MITgcm GGL90 Package

Turbulent Kinetic Energy-Based Vertical Ocean Mixing

Technical Reference: Architecture, Physics, and Implementation

---

Package Location: `MITgcm/pkg/ggl90/`

Primary Reference: Gaspar, P., Y. Gregoris, and J.-M. Lefevre (1990). *A simple eddy kinetic energy model for simulations of the oceanic vertical mixing.* Journal of Geophysical Research, 95(C9), 16,179–16,193.

Implementation Reference: Blanke, B., and P. Delecluse (1993). *Variability of the Tropical Atlantic Ocean Simulated by a General Circulation Model with Two Different Mixed-Layer Physics.* Journal of Physical Oceanography, 23, 1363–1388.

Compiled from source files:

`GGL90.h`	State variable declarations
`GGL90_OPTIONS.h`	Compile-time options
`ggl90_calc.F`	Main driver routine (1,177 lines)
`ggl90_mixinglength.F`	Mixing length computation (421 lines)
`ggl90_calc_diff.F`	Diffusivity interface (74 lines)
`ggl90_calc_visc.F`	Viscosity interface (64 lines)
`ggl90_readparms.F`	Parameter file reader (451 lines)
`ggl90_check.F`	Consistency checks (166 lines)
`ggl90_init_fixed.F`	Time-invariant initialization (67 lines)
`ggl90_init_varia.F`	Time-varying initialization (156 lines)
`ggl90_diagnostics_init.F`	Diagnostics registration (202 lines)
`ggl90_output.F`	Diagnostics and I/O (98 lines)
`ggl90_read_pickup.F`	Restart file reader (69 lines)
`ggl90_write_pickup.F`	Restart file writer (60 lines)
`ggl90_exchanges.F`	Halo exchanges (48 lines)
`ggl90_idemix.F`	IDEMIX internal wave model (598 lines)
`ggl90_add_stokesdrift.F`	Langmuir circulation (79 lines)

Total package size: ∼3,958 lines of Fortran code

Contents

1 Introduction and Scientific Background **6**
1.1 Physical Motivation . . . 6
1.1.1 Key Physical Processes . . . 6
1.2 GGL90 vs. KPP: A Comparison . . . 7
1.2.1 When to Use GGL90 . . . 7
1.2.2 When to Use KPP . . . 8
1.3 Key Equations Overview . . . 8
1.3.1 TKE Evolution Equation . . . 8
1.3.2 Eddy Viscosity and Diffusivity . . . 9
1.3.3 Mixing Length . . . 9
1.3.4 Boundary Conditions . . . 9

2 Governing Equations (overview) **9**
2.1 TKE Evolution Equation . . . 10
2.2 Eddy Viscosity and Diffusivity . . . 10
2.3 Mixing Length . . . 10
2.4 Boundary Conditions . . . 11
2.4.1 Surface . . . 11
2.4.2 Bottom . . . 11
2.5 Coordinate System and Discretization . . . 11

3 Package Architecture and Call Flow **11**
3.1 Package Initialization Sequence . . . 11
3.1.1 Phase 1: Fixed Initialization (`GGL90_INIT_FIXED`) . . . 11
3.1.2 Phase 2: Variable Initialization (`GGL90_INIT_VARIA`) . . . 12
3.2 Per-Timestep Execution Flow . . . 13
3.2.1 Main Driver: `GGL90_CALC` . . . 14
3.3 File Organization . . . 15
3.3.1 File Interdependencies . . . 16
3.3.2 Optional Feature Compilation . . . 17

4 Parameter Initialization **17**
4.1 Core Physics Parameters . . . 17
4.1.1 Physical Interpretation . . . 17
4.2 Mixing Length Control Parameters . . . 19
4.3 Numerical and I/O Parameters . . . 19
4.4 Example Parameter File . . . 19
4.5 Application-Specific Configurations . . . 21
4.6 Parameter Validation . . . 21

5 Main Driver Routine **21**
5.1 Subroutine Signature and Input Parameters . . . 22
5.2 Coordinate System Handling . . . 22
5.3 Initialization of Local Arrays . . . 23
5.4 Interface Thickness Factors . . . 23
5.5 Step-by-Step Execution Summary . . . 24

6 Stratification and Buoyancy **25**
6.1 Buoyancy Frequency Squared (N^2) . . . 25
6.1.1 Implementation . . . 25
6.1.2 Physical Interpretation . . . 25

7 Surface Forcing **26**
7.1 Surface Boundary Condition . . . 26
7.1.1 Physical basis . . . 26
7.1.2 Friction velocity calculation . . . 27
7.1.3 Application of surface boundary condition . . . 28
7.1.4 Surface minimum TKE . . . 29

8 Interior Mixing **30**
8.1 Production Term . . . 30
8.1.1 Vertical shear computation . . . 30
8.1.2 Production term calculation . . . 31

9 Boundary / Surface Layer Mixing **32**
9.1 Initial Estimate from Gaspar et al. (1990) . . . 33
9.1.1 Physical motivation . . . 33
9.1.2 Implementation in `GGL90_CALC` . . . 33
9.1.3 Coordinate transformation . . . 34
9.1.4 Typical values . . . 35
9.2 Method 0: Simple Distance to Boundaries . . . 35
9.2.1 Algorithm . . . 35
9.2.2 Implementation . . . 35
9.2.3 When to use Method 0 . . . 36
9.3 Method 1: One-Way Downward Sweep . . . 36
9.3.1 Algorithm . . . 37
9.3.2 Implementation . . . 37
9.3.3 Limitations . . . 38
9.3.4 When to use Method 1 . . . 38
9.4 Method 2: Two-Way Sweep (Blanke & Delecluse 1993) . . . 38
9.4.1 Physical motivation . . . 39
9.4.2 Algorithm overview . . . 39
9.4.3 Implementation in MITgcm . . . 39
9.4.4 Worked example: Idealized stratification . . . 42
9.4.5 Comparison of Methods 0, 1, and 2 . . . 43

9.5 Surface and Bottom Constraints (`mxlSurfFlag`) . . . 43
9.5.1 Purpose of `mxlSurfFlag` . . . 43
9.5.2 Implementation . . . 44
9.5.3 When to use `mxlSurfFlag` . . . 44
9.5.4 Bottom boundary condition . . . 45
9.5.5 Regularization: `GGL90_REGULARIZE_MIXINGLENGTH` . . . 45
9.6 Langmuir Circulation Enhancement (Optional) . . . 46

10 Core Diffusivity/Prognostic Update **46**
10.1 TKE Evolution and Budget . . . 46
10.2 Governing Equation . . . 46
10.3 Production Term . . . 48
10.3.1 Vertical shear computation . . . 48
10.3.2 Production term calculation . . . 50
10.4 Buoyancy Term . . . 50
10.4.1 Sign convention and physical interpretation . . . 50
10.4.2 Implementation . . . 51
10.5 Dissipation Term . . . 51
10.5.1 Physical basis . . . 51
10.5.2 Implementation: Semi-implicit time integration . . . 52
10.5.3 Explicit/implicit splitting . . . 52
10.6 Vertical Diffusion of TKE . . . 53
10.6.1 Diffusivity of TKE: κ_e . . . 54
10.6.2 Implicit vertical diffusion solver . . . 54
10.6.3 Tridiagonal solve . . . 56
10.7 Time Integration Scheme: Operator Splitting . . . 56
10.7.1 Time step selection . . . 57
10.7.2 Horizontal diffusion of TKE (optional) . . . 57
10.8 Eddy Coefficient Calculation . . . 58
10.9 Eddy Viscosity (κ_m) . . . 58
10.9.1 Theoretical basis . . . 58
10.9.2 Implementation . . . 59
10.9.3 Typical values . . . 60
10.10 TKE Prandtl Number . . . 60
10.10.1 Formulation as a function of Richardson number . . . 60
10.10.2 Implementation . . . 61
10.10.3 Alternative formulation with IDEMIX . . . 62
10.11 Eddy Diffusivity (κ_h) . . . 63
10.11.1 Typical values . . . 63
10.12 Background Floors and Maximum Caps . . . 63
10.12.1 Background viscosity . . . 64
10.12.2 Background diffusivity . . . 64
10.12.3 Coordinate transformation . . . 64
10.12.4 Maximum cap (optional) . . . 64
10.12.5 Masking at dry interfaces . . . 64

11 Additional Mixing Processes **65**
11.1 IDEMIX Extension . . . 65
11.2 Overview of Internal Wave Model . . . 65
11.3 Coupling to GGL90 . . . 65

12 Eddy Coefficients and Model Interface **66**
12.1 `GGL90_CALC_VISC`: Viscosity Transfer . . . 66
12.1.1 Purpose . . . 66
12.1.2 Call signature . . . 66
12.1.3 Implementation . . . 67
12.1.4 Location in time-stepping sequence . . . 68
12.2 `GGL90_CALC_DIFF`: Diffusivity Transfer . . . 68
12.2.1 Purpose . . . 68
12.2.2 Call signature . . . 68
12.2.3 Implementation . . . 68
12.3 `GGL90_EXCHANGES`: Halo Updates . . . 69
12.3.1 Purpose . . . 70
12.3.2 Implementation . . . 70

13 Initialization and Restart **70**
13.1 `GGL90_READPARMS`: Parameter Input . . . 70
13.2 `GGL90_INIT_FIXED`: Time-Invariant Initialization . . . 71
13.3 `GGL90_INIT_VARIA`: Time-Varying Field Initialization . . . 71
13.4 Restart Files: `GGL90_READ/WRITE_PICKUP` . . . 73

14 Compile-Time Options **73**
14.1 Core Options . . . 73
14.2 Optional Feature Flags . . . 74

15 Output and Diagnostics **74**
15.1 Available Diagnostic Fields . . . 74
15.2 Diagnostics Registration . . . 75

16 Package Validation **75**

17 Numerical Considerations & Frequently Encountered Issues **75**
17.1 Parameter Tuning Guidance . . . 76
17.2 Common Mistakes and Solutions . . . 76
17.3 Performance Considerations . . . 76
17.4 Minimum Alpha for Oscillation-Free Solutions . . . 77
17.4.1 Motivation: Why Alpha Matters for Numerical Accuracy . . . 77
17.4.2 Oscillation Diagnosis Results . . . 77
17.4.3 Why ECCOv4 Uses $\alpha = 30$ . . . 78
17.4.4 Practical Recommendations . . . 78

18 Conclusions **79**

19 Glossary of Symbols **80**

A Complete Input/Output Mapping **82**
A.1 External Inputs from Main Model 83
A.2 Package Outputs to Main Model 83
A.3 Internal Prognostic State 83
A.4 Data Flow Diagram 84

1 Introduction and Scientific Background

The GGL90 package implements a one-equation turbulence closure scheme based on a prognostic equation for turbulent kinetic energy (TKE). Originally proposed by Gaspar, Gregoris, and Lefevre (1990, hereafter GGL90), this parameterization provides a physically based representation of vertical mixing throughout the ocean water column. The MITgcm implementation closely follows the Blanke and Delecluse (1993) formulation, which was developed for the OPA ocean model and has been extensively validated in global ocean simulations.

Unlike diagnostic mixing schemes (such as KPP), GGL90 carries TKE as a prognostic variable, explicitly simulating its production, dissipation, and transport. This approach captures the time-dependent evolution of turbulence, making it particularly well-suited for scenarios with rapidly varying forcing or when turbulence memory effects are important.

1.1 Physical Motivation

Vertical mixing in the ocean arises from turbulent motions that span a wide range of spatial and temporal scales. While direct numerical simulation of these turbulent flows is computationally infeasible in global ocean models, turbulence closure schemes provide a practical framework for parameterizing the effects of unresolved turbulent motions on the resolved-scale flow.

GGL90 adopts a **one-equation turbulence closure** approach, wherein:

1. **Turbulent kinetic energy (TKE)** is treated as a prognostic variable satisfying a budget equation that balances production, buoyancy work, dissipation, and diffusion.

2. **Mixing length (L_{mix})** is diagnosed from the vertical structure of TKE and stratification, subject to geometric constraints (distance to boundaries).

3. **Eddy viscosity (κ_m) and diffusivity (κ_h)** are computed from TKE and mixing length using dimensional analysis:
$$\kappa_m \sim L_{\text{mix}} \sqrt{\text{TKE}}$$

4. **Stability functions** relate the eddy diffusivity to eddy viscosity through an empirical Prandtl number that depends on local stratification.

This framework is grounded in the classical turbulence theory of Kolmogorov (1942) and extends it to stratified, sheared oceanic flows where buoyancy effects play a critical role in modulating turbulence.

1.1.1 Key Physical Processes

The GGL90 scheme captures several fundamental mechanisms of oceanic turbulence:

- **Shear production** – Vertical shear in the horizontal velocity field ($S^2 = |\partial \mathbf{u}/\partial z|^2$) generates TKE through the extraction of energy from the mean flow. This mechanism dominates in regions with strong currents or wind-driven surface layers.

- **Buoyancy work** – Stratification ($N^2 = -g/\rho_0\, \partial\rho/\partial z$) either suppresses turbulence (stable stratification, $N^2 > 0$) or enhances it (convective instability, $N^2 < 0$). The buoyancy term in the TKE budget can act as either a sink or source depending on the sign of N^2 and the direction of turbulent buoyancy flux.
- **Dissipation** – Turbulent kinetic energy is irreversibly converted to heat at the smallest (Kolmogorov) scales. GGL90 parameterizes this dissipation rate as $\varepsilon \sim \mathrm{TKE}^{3/2}/L_{\mathrm{mix}}$, consistent with dimensional analysis.
- **Vertical transport** – TKE can be transported vertically by turbulent diffusion and pressure work. GGL90 includes a diffusion term for TKE with diffusivity $\kappa_E \sim \kappa_m$.
- **Surface and bottom forcing** – Wind stress at the ocean surface injects TKE into the boundary layer. Bottom drag can also generate TKE near the seafloor. These boundary sources are critical for maintaining realistic mixing levels.

1.2 GGL90 vs. KPP: A Comparison

Both GGL90 and KPP are widely used in ocean general circulation models, but they differ fundamentally in their approach to parameterizing vertical mixing. Table 1 summarizes the key distinctions.

Table 1: Comparison of GGL90 and KPP vertical mixing schemes.

Aspect	**GGL90**	**KPP**
Type	Prognostic (TKE-based)	Diagnostic (profile-based)
Primary variable	TKE	Boundary layer depth (h_{bl})
Time dependence	Explicit TKE evolution	Instantaneous diagnosis
Mixing length	Geometric + TKE-based	Geometric (distance to h_{bl})
Boundary layer	Emergent from TKE	Explicitly diagnosed
Memory effects	Yes (TKE persists)	No (local diagnosis)
Computational cost	Higher (extra PDE)	Lower (diagnostic)
Calibration	Few parameters	Many profile functions
Deep convection	Natural (TKE grows)	Requires special treatment
Adjoint complexity	Moderate (Appendix **??**)	Higher (many branches)

1.2.1 When to Use GGL90

GGL90 is particularly advantageous in the following scenarios:

- **Rapidly varying forcing** – Diurnal or synoptic-scale wind variations, where TKE memory affects the response time of mixing.
- **Deep convection** – High-latitude regions where convective plumes penetrate to great depths. GGL90 naturally allows TKE to grow and propagate downward without requiring special convective adjustment.

- **Adjoint-based optimization** – GGL90's simpler parameter space facilitates gradient-based data assimilation (see Appendix **??** for detailed discussion).
- **Coupled models** – When the ocean model is coupled to sea ice or atmosphere models with rapid timescale interactions, the prognostic nature of TKE can improve synchronization.

1.2.2 When to Use KPP

KPP remains preferred in many applications due to:

- **Lower computational cost** – No additional PDE to solve.
- **Well-established calibration** – Decades of tuning in numerous ocean models (e.g., CESM, ACCESS, GFDL models).
- **Explicit non-local flux** – KPP includes a counter-gradient term for boundary layer mixing, which GGL90 does not.
- **Smoother mixed layer base** – KPP's explicit h_{bl} can produce sharper transitions than GGL90's gradual TKE decay.

In practice, both schemes can produce realistic simulations when properly tuned. The choice often depends on the specific application, available computational resources, and user familiarity.

1.3 Key Equations Overview

This subsection provides a high-level summary of the governing equations in GGL90. Detailed derivations and code implementations are presented in subsequent sections.

1.3.1 TKE Evolution Equation

The prognostic equation for TKE (denoted $e = \mathrm{TKE}$ in the code) is:

$$\frac{\partial \mathrm{TKE}}{\partial t} = P + B - \varepsilon + D \tag{1}$$

where:

$$\begin{aligned}
P &= \kappa_m \, S^2 && \text{(shear production)} \\
B &= -\kappa_h \, N^2 && \text{(buoyancy term)} \\
\varepsilon &= c_\varepsilon \frac{\mathrm{TKE}^{3/2}}{L_{\mathrm{mix}}} && \text{(dissipation)} \\
D &= \frac{\partial}{\partial z}\left(\kappa_E \frac{\partial \mathrm{TKE}}{\partial z}\right) && \text{(vertical diffusion)}
\end{aligned}$$

The dissipation constant c_ε (denoted `GGL90ceps` in the code) is typically set to $c_\varepsilon \approx 0.7$, following empirical constraints from laboratory and atmospheric boundary layer studies.

1.3.2 Eddy Viscosity and Diffusivity

Eddy coefficients are diagnosed from TKE and mixing length:

$$\kappa_m = c_k \, L_{\text{mix}} \sqrt{\text{TKE}} \tag{2}$$

$$\kappa_h = \frac{\kappa_m}{\alpha} \tag{3}$$

where $c_k \approx 0.1$ (constant `GGL90ck`) and α (constant `GGL90alpha`) is the turbulent Prandtl number. In the original Gaspar et al. (1990) formulation, $\alpha = 1$. However, global ocean applications often use larger values (see Appendix **??** for ECCOv4's $\alpha = 30$ choice) to reduce diffusivity relative to viscosity, thereby maintaining sharp thermoclines.

1.3.3 Mixing Length

The mixing length is diagnosed from TKE and stratification following Gaspar et al. (1990), equation (9):

$$L_{\text{mix}}^{(\text{Gaspar})} = \frac{\sqrt{2}\,\sqrt{\text{TKE}}}{\sqrt{N^2}} \tag{4}$$

This expression is then limited by geometric constraints (distance to surface, distance to bottom) using one of three methods (`mxlMaxFlag` = 0, 1, or 2). The Blanke and Delecluse (1993) Method 2 (two-way sweep) is most commonly used (see Appendix **??**).

1.3.4 Boundary Conditions

Surface ($z = 0$):

$$\kappa_E \frac{\partial \text{TKE}}{\partial z}\bigg|_{z=0} = m_2 \, u_*^3 \tag{5}$$

where $u_* = \sqrt{\tau/\rho_0}$ is the friction velocity and $m_2 \approx 3.75$ (constant `GGL90m2`) determines the fraction of wind energy flux entering the TKE budget.

Bottom ($z = -H$):

$$\text{TKE}\big|_{z=-H} = \max(\text{TKE}_{\text{bottom}}, \text{TKE}_{\text{min}}) \tag{6}$$

where $\text{TKE}_{\text{bottom}}$ (`GGL90TKEbottom`) represents background TKE generated by bottom drag or internal wave breaking.

These equations form the core of the GGL90 parameterization and are solved at every model timestep. The following sections describe their implementation in MITgcm in detail, with extensive code references and examples.

2 Governing Equations (overview)

This section provides a high-level overview of the governing equations for GGL90. Detailed derivations and computational procedures appear in later sections.

2.1 TKE Evolution Equation

The fundamental prognostic equation solved by GGL90 is the turbulent kinetic energy (TKE) evolution equation:

$$\frac{\partial \mathrm{TKE}}{\partial t} + \vec{u} \cdot \nabla \mathrm{TKE} = P + B - \varepsilon + \frac{\partial}{\partial z}\left(\kappa_E \frac{\partial \mathrm{TKE}}{\partial z}\right) \tag{7}$$

where:

- $P = \kappa_m S^2$ — Shear production from vertical velocity gradients
- $B = -\kappa_h N^2$ — Buoyancy flux (positive for unstable stratification)
- $\varepsilon = c_\varepsilon \frac{\mathrm{TKE}^{3/2}}{L}$ — Dissipation rate
- $\kappa_E = \kappa_m$ — Vertical diffusivity of TKE

The TKE field $\mathrm{TKE}(x, y, z, t)$ is the only prognostic variable in GGL90; all mixing coefficients are diagnosed from TKE at each time step.

2.2 Eddy Viscosity and Diffusivity

The eddy viscosity κ_m and diffusivity κ_h are computed from TKE via:

$$\kappa_m(z) = c_k L(z) \sqrt{\mathrm{TKE}(z)} \tag{8}$$

$$\kappa_h(z) = \frac{\kappa_m(z)}{\alpha} \tag{9}$$

where:

- $c_k = 0.1$ — Dimensionless closure constant
- $L(z)$ — Mixing length (stratification-dependent, bounded by distance to boundaries)
- α — Turbulent Prandtl number (ratio κ_m/κ_h; default 1.0, ECCOv4: 30.0)

2.3 Mixing Length

The mixing length $L(z)$ is computed from the Gaspar et al. (1990) formulation:

$$L(z) = \sqrt{2} \frac{\sqrt{\mathrm{TKE}(z)}}{\sqrt{\max(N^2(z), 0)}} \quad \text{(stable stratification)} \tag{10}$$

with hard limits imposed by distance to boundaries:

$$L(z) \leftarrow \min\left(\max\left(L(z), L_{\min}\right), \kappa|z|\right) \tag{11}$$

where $\kappa = 0.4$ is the von Kármán constant and $|z|$ is distance from the surface.

2.4 Boundary Conditions

2.4.1 Surface

At the ocean surface ($z = 0$), TKE is forced by wind stress:

$$\text{TKE}(z = 0) = \max\left(\frac{u_*^2}{B_1}, \text{GGL90TKEsurfMin}\right) \tag{12}$$

where $u_*^2 = \sqrt{(\tau_x/\rho_0)^2 + (\tau_y/\rho_0)^2}$ is the friction velocity squared and $B_1 \approx 16.6$ is a dimensionless coefficient.

2.4.2 Bottom

At the ocean bottom, TKE is set to a minimum background value:

$$\text{TKE}(z = z_{\text{bottom}}) = \max\left(\text{GGL90TKEbottom}, \text{GGL90TKEmin}\right) \tag{13}$$

2.5 Coordinate System and Discretization

MITgcm uses a z-coordinate with $z = 0$ at the surface and $z < 0$ in the ocean interior. The vertical coordinate is discretized into N_z levels with cell-center depths $z_c(k)$ and cell-interface depths $z_f(k)$. GGL90 computes mixing coefficients at cell interfaces (where vertical fluxes are evaluated) and updates TKE at cell centers (where the prognostic equation is integrated).

See §5 for the complete computational sequence and §10 for detailed time-stepping procedures.

3 Package Architecture and Call Flow

The GGL90 package follows MITgcm's modular architecture, with clearly separated initialization, computation, and I/O phases. This section describes the overall package structure, the sequence of subroutine calls, and how GGL90 integrates with the main model timestepping loop.

3.1 Package Initialization Sequence

GGL90 initialization occurs in two phases: **fixed initialization** (once at model startup) and **variable initialization** (which may occur multiple times, e.g., when restarting from a pickup file).

3.1.1 Phase 1: Fixed Initialization (`GGL90_INIT_FIXED`)

Called from `PACKAGES_INIT_FIXED` in the main model initialization. This routine (`ggl90_init_fixed.F:7`) performs time-invariant setup:

```
SUBROUTINE GGL90_INIT_FIXED( myThid )
  ! Purpose: Initialize GGL90 variables that are kept fixed
      during the run

#ifdef ALLOW_DIAGNOSTICS
  IF ( useDiagnostics ) THEN
    CALL GGL90_DIAGNOSTICS_INIT( myThid )  ! Register
        diagnostic fields
  ENDIF
#endif

#ifdef ALLOW_GGL90_SMOOTH
  ! Initialize corner-point mask for horizontal smoothing
  DO bj=myByLo(myThid),myByHi(myThid)
    DO bi=myBxLo(myThid),myBxHi(myThid)
      DO j=1-OLy,sNy+OLy
        DO i=1-OLx,sNx+OLx
          mskCor(i,j,bi,bj) = 1.0
        ENDDO
      ENDDO
      ! Special treatment for cubed-sphere corners
      IF ( useCubedSphereExchange ) THEN
        CALL FILL_CS_CORNER_TR_RL( ... )
      ENDIF
    ENDDO
  ENDDO
#endif

END SUBROUTINE
```

Listing 1: `ggl90_init_fixed.F`, lines 7–44 – Fixed initialization.

Key actions:

- Register diagnostic output fields (TKE, viscosity, diffusivity, etc.) via `GGL90_DIAGNOSTICS_INIT`.
- If horizontal smoothing is enabled (`ALLOW_GGL90_SMOOTH`), initialize corner-point masks for proper averaging at cubed-sphere edges.

3.1.2 Phase 2: Variable Initialization (`GGL90_INIT_VARIA`)

Called from `PACKAGES_INIT_VARIABLES` after parameters are read but before timestepping begins. This routine (`ggl90_init_varia.F:7`) initializes the prognostic TKE field:

```
SUBROUTINE GGL90_INIT_VARIA( myThid )
```

```
2    ! Initialize TKE field either from file or with default
       values
3
4    IF ( GGL90TKEFile .NE. ' ' ) THEN
5      ! Read TKE from binary file
6      CALL READ_FLD_XYZ_RL( GGL90TKEFile, ' ', GGL90TKE, 0,
         myThid )
7    ELSE
8      ! Initialize with minimum TKE
9      DO bj = myByLo(myThid), myByHi(myThid)
10       DO bi = myBxLo(myThid), myBxHi(myThid)
11         DO k = 1, Nr
12           DO j = 1-OLy, sNy+OLy
13             DO i = 1-OLx, sNx+OLx
14               GGL90TKE(i,j,k,bi,bj) = GGL90TKEmin
15             ENDDO
16           ENDDO
17         ENDDO
18       ENDDO
19     ENDDO
20   ENDIF
21
22   ! Initialize IDEMIX fields if enabled
23 #ifdef ALLOW_GGL90_IDEMIX
24   IF ( useIDEMIX ) THEN
25     CALL GGL90_IDEMIX_INIT_VARIA( myThid )
26   ENDIF
27 #endif
28
29 END SUBROUTINE
```

Listing 2: `ggl90_init_varia.F`, lines 42–80 (excerpts) – TKE initialization.

Key actions:

- If `GGL90TKEFile` is specified in `data.ggl90`, read initial TKE from file. Otherwise, initialize to `GGL90TKEmin`.
- If IDEMIX is enabled, initialize internal wave energy fields.
- Exchange halo regions for parallel runs.

3.2 Per-Timestep Execution Flow

At each model timestep, GGL90 is called from the main timestepping loop (`model/src/do_oceanic_phys.F`) as part of the vertical mixing calculations. The top-level entry point is `GGL90_CALC`.

Figure 1 illustrates the complete call sequence.

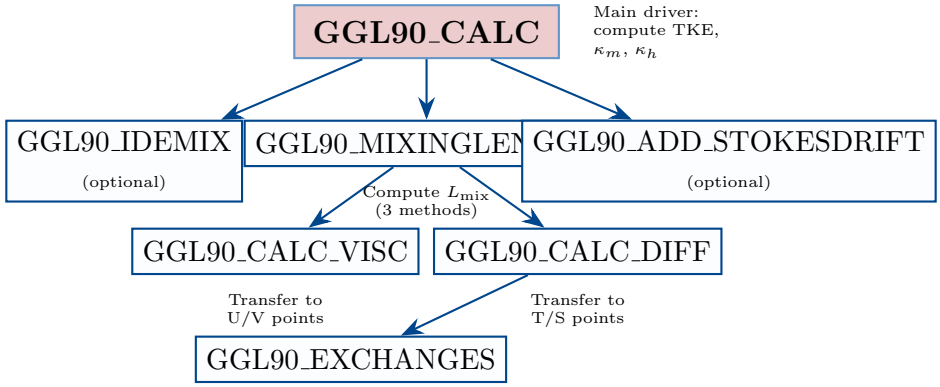

Figure 1: GGL90 call flow diagram showing the hierarchy of subroutines invoked during each timestep. Optional features (IDEMIX, Langmuir) are marked.

3.2.1 Main Driver: `GGL90_CALC`

The primary computational routine (`ggl90_calc.F:13`) orchestrates all GGL90 calculations. Its execution proceeds as follows:

Step 1. Setup (lines 190–260)

- Determine coordinate system (Z-coordinates vs. P-coordinates)
- Initialize local arrays for mixing coefficients, TKE, etc.
- Compute interface thickness factors (`hFacI`)

Step 2. Optional: IDEMIX (lines 259–266)

- If `useIDEMIX = .TRUE.`, call `GGL90_IDEMIX` to compute internal wave energy dissipation (`IDEMIX_gTKE`)
- This provides an additional source term for the TKE budget

Step 3. Compute stratification and shear (lines 300–550)

- Buoyancy frequency squared: $N^2 = -g/\rho_0 \, \partial\rho/\partial z$ (from input `sigmaR`)
- Vertical shear squared: $S^2 = (\partial u/\partial z)^2 + (\partial v/\partial z)^2$
- Surface friction velocity: $u_*^2 = \tau/\rho_0$

Step 4. Optional: Langmuir (lines 450–550)

- If `useLANGMUIR = .TRUE.`, augment shear with Stokes drift contribution

Step 5. Compute mixing length (lines 550–600)

- Call `GGL90_MIXINGLENGTH` to compute L_{mix} using one of three methods (selected by `mxlMaxFlag`)

Step 6. Compute eddy coefficients (lines 600–750)

- $\kappa_m = c_k \, L_{\text{mix}} \sqrt{\max(\text{TKE}, \text{TKE}_{\min})}$
- $\kappa_h = \kappa_m / \alpha$
- Apply background floors and maximum caps

Step 7. TKE budget terms (lines 750–900)

- Production: $P = \kappa_m \, S^2$
- Buoyancy: $B = -\kappa_h \, N^2$
- Dissipation: $\varepsilon = c_\varepsilon \, \text{TKE}^{3/2} / L_{\text{mix}}$

Step 8. Build tridiagonal system (lines 900–1000)

- Set up implicit time-stepping matrix for TKE equation
- Surface BC: wind-driven TKE flux
- Bottom BC: minimum TKE or drag-generated TKE

Step 9. Solve for TKE (lines 1000–1050)

- Solve tridiagonal system using Thomas algorithm
- Update `GGL90TKE` array

Step 10. Optional: Horizontal smoothing (lines 1050–1100)

- If `ALLOW_GGL90_SMOOTH` is defined, apply horizontal averaging (OPA method) to reduce grid-scale noise

Step 11. Transfer to model arrays (lines 1100–1150)

- `GGL90_CALC_VISC`: Transfer κ_m to U/V-point arrays (`GGL90viscArU`, `GGL90viscArV`)
- `GGL90_CALC_DIFF`: Transfer κ_h to T/S-point array (`GGL90diffKr`)

Step 12. Diagnostics (lines 1150–1177)

- Fill diagnostic arrays if requested (e.g., TKE snapshots, mixing length, viscosity profiles)

This 12-step sequence is executed once per timestep for each horizontal tile (in parallel domain decomposition).

3.3 File Organization

The GGL90 package consists of 23 source files totaling approximately 3,958 lines of Fortran code. Table 2 summarizes the purpose of each file.

Table 2: Summary of GGL90 source files with line counts and descriptions.

File	Description	Lines
Core Computation		
`ggl90_calc.F`	Main driver, TKE budget, coefficient calc.	1,177
`ggl90_mixinglength.F`	Mixing length (3 methods)	421
`ggl90_calc_diff.F`	Transfer diffusivity to T/S points	74
`ggl90_calc_visc.F`	Transfer viscosity to U/V points	64
Configuration		
`GGL90.h`	Common blocks, variable declarations	178
`GGL90_OPTIONS.h`	Compile-time CPP flags	42
`ggl90_readparms.F`	Read parameters from data.ggl90	451
`ggl90_check.F`	Validate parameter consistency	166
Initialization		
`ggl90_init_fixed.F`	Time-invariant initialization	67
`ggl90_init_varia.F`	Time-varying initialization (TKE)	156
I/O and Diagnostics		
`ggl90_diagnostics_init.F`	Register diagnostic fields	202
`ggl90_output.F`	Output state to files	98
`ggl90_read_pickup.F`	Read restart (pickup) files	69
`ggl90_write_pickup.F`	Write restart (pickup) files	60
Optional Features		
`ggl90_idemix.F`	IDEMIX internal wave model	598
`ggl90_add_stokesdrift.F`	Langmuir circulation	79
Utilities		
`ggl90_exchanges.F`	Halo exchanges for parallel runs	48
`ggl90_ad_*.h`	Adjoint model directives (4 files)	8
Total		**3,958**

3.3.1 File Interdependencies

- **Header files** (`GGL90.h`, `GGL90_OPTIONS.h`) are included by all computational routines to access parameters and state variables.
- **Parameter reading** (`ggl90_readparms.F`) must occur before `ggl90_init_varia.F` because initialization depends on parameter values (e.g., `GGL90TKEmin`).
- **Diagnostics initialization** (`ggl90_diagnostics_init.F`) must occur before the first timestep to register output fields with MITgcm's diagnostics package.
- **Main driver** (`ggl90_calc.F`) calls helper routines (`ggl90_mixinglength.F`, `ggl90_calc_diff.F`, `ggl90_calc_visc.F`) but does not depend on I/O routines.

3.3.2 Optional Feature Compilation

Several features are controlled by CPP flags defined in `GGL90_OPTIONS.h` (see Section 14 for details):

- `ALLOW_GGL90_IDEMIX`: Enables `ggl90_idemix.F` (internal wave forcing).
- `ALLOW_GGL90_LANGMUIR`: Enables `ggl90_add_stokesdrift.F` (Langmuir turbulence).
- `ALLOW_GGL90_SMOOTH`: Enables horizontal smoothing in `ggl90_calc.F`.
- `ALLOW_GGL90_HORIZDIFF`: Enables horizontal diffusion of TKE.

If these flags are undefined, the corresponding code blocks are excluded at compile time, reducing memory footprint and computational cost.

4 Parameter Initialization

GGL90 parameters are read from the namelist file `data.ggl90` during model initialization. The reading is handled by `ggl90_readparms.F` (lines 6–451), which sets default values and then overwrites them with user-specified values if present. This section documents all GGL90 parameters with their default values, physical meanings, and typical ranges.

Parameters are organized into three namelist groups:

- `GGL90_PARM01`: Core GGL90 parameters (lines 48–57)
- `GGL90_PARM02`: IDEMIX parameters (optional, lines 63–68)
- `GGL90_PARM03`: Langmuir parameters (optional, lines 75–76)

4.1 Core Physics Parameters

Table 3 lists the fundamental parameters governing TKE dynamics, eddy coefficients, and mixing length computation.

4.1.1 Physical Interpretation

- c_k **and** c_ε – These constants arise from dimensional analysis and are calibrated against laboratory experiments, large-eddy simulations, and observations. Their default values (0.1 and 0.7) are well-established in the turbulence closure literature.
- α **(Prandtl number)** – In neutral boundary layers, momentum and heat are transported similarly ($\alpha \approx 1$). However, in the stratified ocean interior, $\alpha > 1$ is often needed to prevent excessive tracer diffusion. The choice of α is application-specific and often tuned to match observed mixed layer depths and thermocline sharpness.
- m_2 **(surface flux fraction)** – Not all wind energy input to the ocean surface is converted to TKE. Some goes into surface gravity waves and Langmuir circulation. $m_2 \approx 3.75$ implies that a few percent of wind power enters the TKE budget.

Table 3: Core physics parameters for GGL90 (from `ggl90_readparms.F:108--120`).

Parameter	Default	Units	Description
`GGL90ck`	0.1	–	Constant c_k in eddy viscosity formula $\kappa_m = c_k L_{\text{mix}} \sqrt{\text{TKE}}$ (eq. 10 of Gaspar et al. 1990). Dimensionless.
`GGL90ceps`	0.7	–	Dissipation constant c_ε in $\varepsilon = c_\varepsilon \text{TKE}^{3/2}/L_{\text{mix}}$. Based on Kolmogorov (1942) theory.
`GGL90alpha`	1.0	–	Turbulent Prandtl number relating diffusivity to viscosity: $\kappa_h = \kappa_m/\alpha$. Gaspar et al. use $\alpha = 1$. See Appendix **??** for alternative values in global applications.
`GGL90m2`	3.75	–	Surface TKE flux constant m_2 in boundary condition $\kappa_E \partial \text{TKE}/\partial z|_{z=0} = m_2 u_*^3$. Value from Blanke & Delecluse (1993).
`GGL90TKEmin`	10^{-11}	m^2/s^2	Minimum TKE enforced throughout the water column. Represents background mixing from internal waves and other unresolved processes.
`GGL90TKEsurfMin`	10^{-4}	m^2/s^2	Minimum TKE at surface. Higher than interior `GGL90TKEmin` to ensure adequate surface boundary layer mixing. Value from Blanke & Delecluse.
`GGL90TKEbottom`	`UNSET_RL`	m^2/s^2	Bottom TKE. If unset, defaults to `GGL90TKEmin`. Can be increased to represent bottom drag or rough topography.
`GGL90mixingLengthMin`	10^{-8}	m	Minimum mixing length. Prevents division by zero and provides regularization. Physically represents molecular scales.
`GGL90viscMax`	10^2	m^2/s	Maximum eddy viscosity. Caps unrealistic values in poorly resolved regions or during convective events.
`GGL90diffMax`	10^2	m^2/s	Maximum eddy diffusivity. Similar purpose as `GGL90viscMax`.

- **Minimum TKE values** – Even in quiescent, stratified regions, some level of mixing persists due to internal wave breaking, double diffusion, and other sub-grid processes. `GGL90TKEmin` ensures that κ_m and κ_h never vanish completely, which would be unphysical and numerically problematic.

4.2 Mixing Length Control Parameters

The computation of mixing length is controlled by `mxlMaxFlag` (lines 121–123) and related flags. Table 4 summarizes these.

Table 4: Mixing length control parameters.

Parameter	Default	Description
`mxlMaxFlag` 0 = Distance to boundaries (simplest) 1 = One-way downward sweep 2 = Two-way sweep (Blanke & Delecluse 1993) Method 2 is recommended (see Appendix **??**).	0	Method for limiting mixing length:
`adMxlMaxFlag`	`UNSET_I`	Alternative `mxlMaxFlag` for adjoint runs. unset, uses `mxlMaxFlag`. See Appendix **?** for adjoint-specific settings.
`mxlSurfFlag`	`.FALSE.`	If `.TRUE.`, forces mixing length between su face and first level to equal the layer thick ness. Ensures adequate surface boundar layer mixing.

Mixing Length Method Comparison:

- **Method 0** – Simplest: $L_{\text{mix}} = \min(L_{\text{Gaspar}}, z_{\text{surf}}, z_{\text{bot}})$ where z_{surf} and z_{bot} are distances to surface and bottom. Fast but can produce discontinuities.
- **Method 1** – Downward sweep smooths L_{mix} in the vertical: starting from the surface, ensures $L_{\text{mix}}(k) \leq L_{\text{mix}}(k-1) + \Delta z$. Prevents abrupt jumps but is asymmetric.
- **Method 2** – Two-way sweep (downward then upward) produces the smoothest profiles and is most physical. Used in OPA and many global applications (Appendix **??**). Adds minimal computational cost. **Recommended for production runs.**

4.3 Numerical and I/O Parameters

Table 5 lists parameters controlling numerics, I/O, and optional features.

4.4 Example Parameter File

A typical `data.ggl90` file for a global ocean simulation might look like:

```
&GGL90_PARM01
# Core physics parameters
  GGL90ck               = 0.1,
  GGL90ceps             = 0.7,
```

Table 5: Numerical and I/O parameters (from `ggl90_readparms.F`).

Parameter	Default	Units/Type	Description
`GGL90diffTKEh`	0.0	m^2/s	Horizontal diffusivity for TKE. Only active if `ALLOW_GGL90_HORIZDIFF` is defined. Usually kept at zero.
`GGL90_dirichlet`	`.TRUE.`	logical	Use Dirichlet boundary conditions for TKE at boundaries. If `.FALSE.`, uses Neumann conditions.
`calcMeanVertShear`	`.FALSE.`	logical	Calculate mean vertical shear at cell centers instead of shear of mean velocity. Affects S^2 computation.
`GGL90TKEFile`	`' '`	string	Binary file containing initial TKE field. If blank, initializes to `GGL90TKEmin`.
`GGL90dumpFreq`	`dumpFreq`	s	Output frequency for GGL90 state files (matches main model by default).
`GGL90mixingMaps`	`.FALSE.`	logical	Output mixing diagnostics to standard out.
`GGL90writeState`	`.FALSE.`	logical	Write GGL90 state to separate output files.
`useIDEMIX`	`.FALSE.`	logical	Enable IDEMIX internal wave model (see Section 11.1).
`useLANGMUIR`	`.FALSE.`	logical	Enable Langmuir circulation parameterization.

```
  GGL90alpha           = 30.0,      # ECCOv4 value (default is 1.0)
  GGL90m2              = 3.75,

# TKE thresholds
  GGL90TKEmin          = 1.0e-7,    # ECCOv4 value (default 1.0e-11)
  GGL90TKEsurfMin      = 1.0e-4,
  GGL90TKEbottom       = 1.0e-6,    # ECCOv4 value

# Mixing length
  GGL90mixingLengthMin = 1.0e-8,
  mxlMaxFlag           = 2,          # Two-way sweep (recommended)
```

```
  mxlSurfFlag           = .TRUE.,     # Force surface mixing

# Caps
  GGL90viscMax          = 1.0e2,
  GGL90diffMax          = 1.0e2,

# I/O
  GGL90dumpFreq         = 86400.0,    # Daily output
/
```

Listing 3: Example `data.ggl90` parameter file.

4.5 Application-Specific Configurations

GGL90 has been successfully deployed in a variety of ocean modeling applications, from idealized process studies to global data-assimilating state estimates. While the default parameter values (based on Gaspar et al. 1990 and Blanke & Delecluse 1993) provide a reasonable starting point, certain applications benefit from targeted parameter adjustments.

For example, global ocean state estimation systems using adjoint-based optimization often employ non-default settings to balance thermocline sharpness, deep ocean mixing representation, and gradient quality. Appendix **??** documents the complete ECCOv4 Release 4 configuration as a well-validated example for $\sim 1^\circ$ global applications.

4.6 Parameter Validation

After reading `data.ggl90`, the routine `ggl90_check.F` (lines 7–166) validates parameter consistency. It checks for:

- **Positive values**: $c_k, c_\varepsilon, \alpha, m_2, \mathrm{TKE}_{\min}$ must all be > 0.
- **Valid flags**: `mxlMaxFlag` $\in \{0, 1, 2\}$.
- **Implicit timestepping**: `implicitDiffusion` and `implicitViscosity` must be `.TRUE.` in `data` (required for stability of TKE equation).
- **IDEMIX requirements**: If `useIDEMIX = .TRUE.`, checks that overlap regions (OLx, OLy) are ≥ 3 for horizontal operators.

If any check fails, the model stops with a descriptive error message. This prevents common configuration mistakes.

5 Main Driver Routine

The subroutine `GGL90_CALC` (`ggl90_calc.F:13--1177`) is the computational heart of the GGL90 package. It is called once per timestep for each tile in the domain decomposition. This section provides a detailed walkthrough of its execution, with code excerpts and line number references.

5.1 Subroutine Signature and Input Parameters

```
SUBROUTINE GGL90_CALC(
     I       bi, bj, sigmaR, myTime, myIter, myThid )

  ! Input parameters:
  !   bi, bj    :: Current tile indices
  !   sigmaR    :: Vertical gradient of iso-neutral density (3D)
  !   myTime    :: Current time in simulation
  !   myIter    :: Current time-step number
  !   myThid    :: Thread ID number

  ! Global variables updated:
  !   GGL90TKE(i,j,k,bi,bj)      :: Turbulent kinetic energy (m
     ^2/s^2)
  !   GGL90viscArU(i,j,k,bi,bj) :: Eddy viscosity at U-points (
     m^2/s)
  !   GGL90viscArV(i,j,k,bi,bj) :: Eddy viscosity at V-points (
     m^2/s)
  !   GGL90diffKr(i,j,k,bi,bj)  :: Eddy diffusivity at T/S
     points (m^2/s)
```

Listing 4: `ggl90_calc.F`, lines 13–63 – Subroutine interface.

Key input: The array `sigmaR` contains the vertical gradient of potential density referenced to the local pressure. This is computed by the main model's equation of state and passed to GGL90. From `sigmaR`, the buoyancy frequency squared (N^2) is derived as:

$$N^2 = -\frac{g}{\rho_0}\frac{\partial \rho}{\partial z} = -\frac{g}{\rho_0}\sigma_R$$

where g is gravitational acceleration and ρ_0 is reference density.

5.2 Coordinate System Handling

MITgcm supports both Z-coordinates (depth, $z < 0$) and P-coordinates (pressure, $p > 0$). GGL90 must handle both conventions. Lines 198–209:

```
IF ( usingPCoords ) THEN
   kSrf = Nr       ! Surface at highest k index
   kTop = Nr
ELSE
   kSrf = 1        ! Surface at k=1 (Z-coordinates)
   kTop = 2
ENDIF
deltaTloc = dTtracerLev(kSrf)  ! Timestep for tracer level

```

```
coordFac = 1.0
IF ( usingPCoords ) coordFac = gravity * rhoConst
recip_coordFac = 1.0 / coordFac
```

Listing 5: `ggl90_calc.F`, lines 198–209 – Coordinate system setup.

Coordinate scaling factor: In P-coordinates, vertical derivatives involve pressure gradients. The factor `coordFac` $= g\rho_0$ converts pressure units to depth-equivalent units. This ensures that mixing length and TKE budget terms have consistent dimensions regardless of coordinate choice.

5.3 Initialization of Local Arrays

Lines 268–300 initialize working arrays to safe default values:

```
DO k=1,Nr
  DO j=1-OLy,sNy+OLy
    DO i=1-OLx,sNx+OLx
      rMixingLength(i,j,k)      = 0.0
      GGL90visctmp(i,j,k)       = 0.0
      KappaE(i,j,k)             = 0.0
      TKEPrandtlNumber(i,j,k)   = 1.0
      GGL90mixingLength(i,j,k)  = GGL90mixingLengthMin
      Nsquare(i,j,k)            = 0.0
      SQRTTKE(i,j,k)            = 0.0
    ENDDO
  ENDDO
ENDDO
DO j=1-OLy,sNy+OLy
  DO i=1-OLx,sNx+OLx
    KappaM(i,j)          = 0.0
    uStarSquare(i,j)     = 0.0
    verticalShear(i,j)   = 0.0
  ENDDO
ENDDO
```

Listing 6: `ggl90_calc.F`, lines 268–300 (excerpts) – Array initialization.

Initializing arrays is essential for robustness, especially in regions where topography creates "dry" cells (masked points).

5.4 Interface Thickness Factors

Lines 242–257 compute thickness factors at cell interfaces (`hFacI`), which are needed for vertical operators:

```
DO k=1,Nr
```

```
2     km1 = MAX(k-1,1)
3     DO j=1-OLy,sNy+OLy
4       DO i=1-OLx,sNx+OLx
5         hFac = MIN( 0.5, _hFacC(i,j,km1,bi,bj) )
6      &           + MIN( 0.5, _hFacC(i,j,k,  bi,bj) )
7         recip_hFacI(i,j,k) = 0.0
8         IF ( hFac .NE. 0.0 )
9      &       recip_hFacI(i,j,k) = 1.0 / hFac
10#ifdef ALLOW_GGL90_IDEMIX
11        hFacI(i,j,k) = hFac
12#endif
13      ENDDO
14    ENDDO
15ENDDO
```

Listing 7: `ggl90_calc.F`, lines 242–257 – Interface thickness calculation.

Explanation: The interface at k lies between cell centers $k-1$ and k. The thickness factor `hFacI(k)` is the average of the two adjacent cell fractions, capped at 0.5 each. This ensures proper handling of partial cells near topography.

5.5 Step-by-Step Execution Summary

The main computational loop (lines 300–1177) proceeds as follows. Each step is detailed in subsequent sections of this document.

Step 1. Compute buoyancy frequency squared (N^2) – From `sigmaR`. See Section **??**.

Step 2. Compute vertical shear squared (S^2) – From velocity fields U and V. See Section **??**.

Step 3. Compute surface friction velocity (u_*^2) – From wind stress. See Section **??**.

Step 4. Compute mixing length (L_{mix}) – Call `GGL90_MIXINGLENGTH` with method selected by `mxlMaxFlag`. See Section **??**.

Step 5. Compute eddy coefficients (κ_m, κ_h) – From TKE and mixing length. See Section 10.8.

Step 6. Compute TKE budget terms – Production, buoyancy, dissipation. See Section **??**.

Step 7. Build implicit matrix – Set up tridiagonal system for TKE equation. See Section **??**.

Step 8. Apply boundary conditions – Surface flux and bottom TKE. See Section **??**.

Step 9. Solve tridiagonal system – Advance TKE to next timestep. See Section **??**.

Step 10. Transfer to model arrays – Call `GGL90_CALC_VISC` and `GGL90_CALC_DIFF`. See Section 12.

The remainder of this document examines each step in detail with code snippets and physical interpretation.

6 Stratification and Buoyancy

6.1 Buoyancy Frequency Squared (N^2)

The buoyancy frequency squared (also called the Brunt-Väisälä frequency squared) quantifies the strength of stable stratification:

$$N^2 = -\frac{g}{\rho_0}\frac{\partial \rho}{\partial z}$$

where g is gravitational acceleration, ρ_0 is a reference density, and $\partial\rho/\partial z$ is the vertical density gradient. Positive N^2 indicates stable stratification (density decreases upward), while negative N^2 indicates convective instability.

6.1.1 Implementation

In MITgcm, the equation of state computes `sigmaR`, the vertical gradient of potential density referenced to local pressure. GGL90 converts this to N^2 (lines 346–348):

```
1 ! Loop over k=2,Nr (interfaces)
2 Nsquare(i,j,k) = gravity * gravitySign * recip_rhoConst
3      &           * sigmaR(i,j,k) * coordFac
```

Listing 8: `ggl90_calc.F`, lines 346–348 – Buoyancy frequency squared.

Notes:

- `gravitySign` $= +1$ for normal gravity, -1 if gravity is reversed (e.g., for testing).
- `recip_rhoConst` $= 1/\rho_0$ where $\rho_0 \approx 1035$ kg/m^3 is the reference density.
- `coordFac` accounts for coordinate system (1.0 for Z-coords, $g\rho_0$ for P-coords).
- N^2 is defined at interfaces (between levels $k-1$ and k), matching the staggering of TKE.

6.1.2 Physical Interpretation

- **Stable stratification** ($N^2 > 0$): Buoyancy acts as a restoring force, suppressing vertical motion. Turbulence is damped, and mixing is weak.
- **Neutral stratification** ($N^2 \approx 0$): Density is vertically uniform. Turbulence can develop freely if shear is present.

- **Convective instability** ($N^2 < 0$): Denser water overlies lighter water. Buoyancy drives overturning motions, generating TKE and enhancing mixing. This occurs during winter cooling, brine rejection from sea ice, or evaporative heat loss.

  In the TKE budget, N^2 enters through the buoyancy term:

$$B = -\kappa_h N^2$$

When $N^2 < 0$, $B > 0$ (source of TKE). When $N^2 > 0$ and mixing transports light water downward, $B < 0$ (sink of TKE).

7 Surface Forcing

Surface forcing in GGL90 is applied via wind stress, which generates turbulent kinetic energy at the ocean-atmosphere interface.

7.1 Surface Boundary Condition

7.1.1 Physical basis

At the ocean surface, TKE is generated by wind stress and surface wave breaking. The standard boundary condition relates the near-surface TKE to the friction velocity u_*:

$$e_{\text{surface}} = B_0 \, u_*^2 \tag{14}$$

where:

- u_*: Friction velocity (m s^{-1}), defined by the surface wind stress τ: $u_*^2 = |\tau|/\rho_0$.
- B_0 (`GGL90m2`): Dimensionless constant relating surface TKE to u_*^2. The default value is $B_0 = 3.75$, based on atmospheric boundary layer measurements and the Blanke & Delecluse (1993) calibration for ocean applications.

Physical interpretation. The factor $B_0 = 3.75$ indicates that the TKE at the surface is several times larger than the mean kinetic energy of the wind-driven current ($\sim u_*^2$). This excess energy comes from surface wave motion and wave breaking, which inject TKE directly into the upper ocean. The calibrated value of 3.75 balances the need for adequate surface layer mixing against observations of mixed layer depth and SST evolution.

Minimum surface TKE. To prevent the surface TKE from vanishing under calm conditions (which would cause numerical issues), GGL90 imposes a minimum value:

$$e_{\text{surface}} = \max(e_{\text{surf,min}}, \, B_0 u_*^2) \tag{15}$$

where $e_{\text{surf,min}}$ = `GGL90TKEsurfMin` is typically 10^{-4} to 10^{-6} m^2 s^{-2} (corresponding to a turbulent velocity scale of 1–10 mm s^{-1}).

7.1.2 Friction velocity calculation

The friction velocity u_* is computed from the surface wind stress components τ_x, τ_y:

$$u_* = \sqrt{\frac{|\tau|}{\rho_0}} = \sqrt{\frac{\sqrt{\tau_x^2 + \tau_y^2}}{\rho_0}} \tag{16}$$

In MITgcm, the surface stresses are provided by the fields `surfaceForcingU` and `surfaceForcingV`, which have units of $\mathrm{m}^2\ \mathrm{s}^{-2}$ (kinematic stress = stress / density). Therefore, the code computes:

$$u_*^2 = \sqrt{\tau_x^2 + \tau_y^2} \tag{17}$$

(Note: MITgcm's `surfaceForcingU/V` are already divided by ρ_0, so no additional normalization is needed.)

The implementation (lines 764–795 of `ggl90_calc.F`) offers two options, controlled by `calcMeanVertShear`:

Option 1: Average of four components (`calcMeanVertShear = .TRUE.`)

```
      IF ( calcMeanVertShear ) THEN
C     by averaging (@ grid-cell center) the 4 components @ U,V pos.
       DO j=jMin,jMax
        DO i=iMin,iMax
         tempU  = surfaceForcingU( i  ,j,bi,bj)
         tempUp = surfaceForcingU(i+1,j,bi,bj)
         tempV  = surfaceForcingV(i, j  ,bi,bj)
         tempVp = surfaceForcingV(i,j+1,bi,bj)
         uStarSquare(i,j) =
     &        ( tempU*tempU + tempUp*tempUp
     &        + tempV*tempV + tempVp*tempVp
     &        )*halfRL
        ENDDO
       ENDDO
```

Listing 9: (ggl90_calc.F:L764–783) Surface friction velocity: Option 1

This averages the squared stresses from the four surrounding U- and V-points, which is appropriate for the Arakawa C-grid staggering.

Option 2: Shear of averaged stress (`calcMeanVertShear = .FALSE.`)

```
      ELSE
       DO j=jMin,jMax
        DO i=iMin,iMax
C     estimate friction velocity uStar from surface forcing
         uStarSquare(i,j) =
     &     ( .5 _d 0*( surfaceForcingU(i,  j,  bi,bj)
```

```
7       &                 + surfaceForcingU(i+1,j,   bi,bj) ) )**2
8       &   + ( .5 _d 0*( surfaceForcingV(i,   j,   bi,bj)
9       &                 + surfaceForcingV(i,   j+1,bi,bj) ) )**2
10         ENDDO
11        ENDDO
12       ENDIF
```

Listing 10: (ggl90_calc.F:L784–795) Surface friction velocity: Option 2

This averages the stresses first, then computes the squared magnitude. The difference is typically small.

After computing `uStarSquare`, the code takes its square root and applies coordinate scaling (lines 864–873):

```
1       DO j=jMin,jMax
2        DO i=iMin,iMax
3 #ifdef ALLOW_AUTODIFF
4          IF ( uStarSquare(i,j) .GT. zeroRL )
5       &       uStarSquare(i,j) = SQRT(uStarSquare(i,j))*
           recip_coordFac
6 #else
7          uStarSquare(i,j) = SQRT(uStarSquare(i,j))*
             recip_coordFac
8 #endif
9        ENDDO
10      ENDDO
```

Listing 11: (ggl90_calc.F:L864–873) Square root and coordinate transformation

Despite the variable name, `uStarSquare` after this step actually contains u_* (not u_*^2), scaled by `recip_coordFac` for pressure coordinates.

7.1.3 Application of surface boundary condition

The surface TKE is imposed as a Dirichlet boundary condition in the implicit TKE solve. The implementation differs slightly between z-coordinates and pressure coordinates.

Z-coordinates (surface at $k = 1$):

```
1       DO j=jMin,jMax
2        DO i=iMin,iMax
3 #ifdef ALLOW_SHELFICE
4          IF ( useShelfIce ) THEN
5           kSrf = MAX(1,kTopC(i,j,bi,bj))
6           kTop = MIN(kSrf+1,Nr)
7          ENDIF
8 #endif
9          GGL90TKE(i,j,kSrf,bi,bj) = maskC(i,j,kSrf,bi,bj)
```

```
10        &              *MAX(GGL90TKEsurfMin,GGL90m2*uStarSquare(i,j))
11            GGL90TKE(i,j,kTop,bi,bj) = GGL90TKE(i,j,kTop,bi,bj)
12        &              - a3d(i,j,kTop)*GGL90TKE(i,j,kSrf,bi,bj)
13            a3d(i,j,kTop) = 0. _d 0
14          ENDDO
```

Listing 12: (ggl90_calc.F:L899–912) Surface BC in z-coordinates

Key details:

- Line 907–908: Set the surface cell TKE to $\max(e_{\text{surf,min}}, B_0 u_*^2)$.
- Lines 909–911: Subtract the contribution of the surface TKE from the RHS of the implicit equation for the second cell (since the surface TKE is now known and moves to the RHS).
- Line 911: Zero out the lower diagonal coefficient for the second cell, effectively decoupling it from the surface cell in the tridiagonal solve.

Pressure coordinates (surface at $k = N_r$):

```
1         IF ( usingPCoords ) THEN
2          DO j=jMin,jMax
3           DO i=iMin,iMax
4            GGL90TKE(i,j,kSrf,bi,bj) = GGL90TKE(i,j,kSrf,bi,bj)
5        &              - c3d(i,j,kSrf) * maskC(i,j,kSrf,bi,bj)
6        &              *MAX(GGL90TKEsurfMin,GGL90m2*uStarSquare(i,j))
7            c3d(i,j,kSrf) = 0. _d 0
8           ENDDO
9          ENDDO
```

Listing 13: (ggl90_calc.F:L883–897) Surface BC in pressure coordinates

The logic is similar, but the upper diagonal `c3d` (instead of lower diagonal `a3d`) is zeroed out, reflecting the reversed vertical indexing in pressure coordinates.

7.1.4 Surface minimum TKE

The parameter `GGL90TKEsurfMin` (default 10^{-4} m^2 s^{-2}) ensures that the surface TKE never vanishes, even under zero wind forcing. This is essential for:

- **Numerical stability**: Prevents division by zero in the mixing length and eddy coefficient calculations.
- **Physical realism**: Even under calm conditions, there is residual turbulence from diurnal heating cycles, residual wave motion, and surface roughness.

Typical values: $e_{\text{surf,min}} = 10^{-6}$ m^2 s^{-2} (very quiescent) to 10^{-4} m^2 s^{-2} (moderate background turbulence).

8 Interior Mixing

Interior mixing in GGL90 is driven by shear production from vertical velocity gradients and buoyancy forcing.

8.1 Production Term

The shear production term $P = \kappa_m S^2$ represents the extraction of kinetic energy from the mean flow by turbulent Reynolds stresses.

8.1.1 Vertical shear computation

The squared vertical shear S^2 is computed in `GGL90_CALC` immediately after the mixing length calculation. MITgcm provides two options for computing S^2, controlled by the run-time parameter `calcMeanVertShear`:

Option 1: Average of four components (`calcMeanVertShear = .TRUE.`) Compute the shear at each of the four surrounding velocity points (two U-points, two V-points), then average:

$$S^2(i,j,k) = \frac{1}{2}\left[\left(\frac{\Delta u_1}{\Delta z}\right)^2 + \left(\frac{\Delta u_2}{\Delta z}\right)^2 + \left(\frac{\Delta v_1}{\Delta z}\right)^2 + \left(\frac{\Delta v_2}{\Delta z}\right)^2\right] \tag{18}$$

where $\Delta u_1 = u(i,j,k-1) - u(i,j,k)$, $\Delta u_2 = u(i+1,j,k-1) - u(i+1,j,k)$, and similarly for v.

```
1         IF ( calcMeanVertShear ) THEN
2  C      by averaging (@ grid-cell center) the 4 vertical shear
       compon @ U,V pos.
3         DO j=jMin,jMax
4          DO i=iMin,iMax
5            tempU  = ( uVel( i  ,j,km1,bi,bj) - uVel( i  ,j,k,bi,bj
                 ) )
6            tempUp = ( uVel(i+1,j,km1,bi,bj) - uVel(i+1,j,k,bi,bj
                 ) )
7            tempV  = ( vVel(i, j  ,km1,bi,bj) - vVel(i, j  ,k,bi,bj
                 ) )
8            tempVp = ( vVel(i,j+1,km1,bi,bj) - vVel(i,j+1,k,bi,bj
                 ) )
9            verticalShear(i,j) = (
10     &                    ( tempU*tempU + tempUp*tempUp )
11     &                  + ( tempV*tempV + tempVp*tempVp )
12     &                              )*halfRL*recip_drC(k)*recip_drC(
           k)
13     &                               *coordFac*coordFac
```

```
14            ENDDO
15           ENDDO
```

Listing 14: (ggl90_calc.F:L468–482) Vertical shear: Option 1

Option 2: Shear of averaged flow (`calcMeanVertShear = .FALSE.`) First average the velocity components to the cell center, then compute shear:

$$S^2(i,j,k) = \left(\frac{\bar{u}(k-1) - \bar{u}(k)}{\Delta z}\right)^2 + \left(\frac{\bar{v}(k-1) - \bar{v}(k)}{\Delta z}\right)^2 \tag{19}$$

where $\bar{u}(k) = \frac{1}{2}[u(i,j,k) + u(i+1,j,k)]$ and $\bar{v}(k) = \frac{1}{2}[v(i,j,k) + v(i,j+1,k)]$.

```
1         ELSE
2 C     from the averaged flow at grid-cell center (2 compon x 2
      pos.)
3         DO j=jMin,jMax
4          DO i=iMin,iMax
5           tempU = ( ( uVel(i,j,km1,bi,bj) + uVel(i+1,j,km1,bi,
              bj) )
6      &              -( uVel(i,j,k  ,bi,bj) + uVel(i+1,j,k  ,bi,
              bj) )
7      &             )*halfRL*recip_drC(k)
8      &              *coordFac
9           tempV = ( ( vVel(i,j,km1,bi,bj) + vVel(i,j+1,km1,bi,
              bj) )
10     &              -( vVel(i,j,k  ,bi,bj) + vVel(i,j+1,k  ,bi,
              bj) )
11     &             )*halfRL*recip_drC(k)
12     &              *coordFac
13          verticalShear(i,j) = tempU*tempU + tempV*tempV
14         ENDDO
15        ENDDO
16       ENDIF
```

Listing 15: (ggl90_calc.F:L483–498) Vertical shear: Option 2

Which option to use? Option 1 is more accurate on the Arakawa C-grid, as it avoids spurious cancellation of checkerboard modes. Option 2 is slightly cheaper computationally. The difference is typically small in well-resolved simulations. Global applications typically use Option 1 (`calcMeanVertShear = .TRUE.`, see Appendix **??**).

8.1.2 Production term calculation

Once S^2 and κ_m are known, the production term is straightforward:

```
1           GGL90TKE(i,j,k,bi,bj) = GGL90TKE(i,j,k,bi,bj)
2       &           + deltaTloc*(
3       &           + KappaM(i,j)*verticalShear(i,j)
4       &           - KappaH*Nsquare(i,j,k)
5       &           - TKEdissipation
6       &           )
```

Listing 16: (ggl90_calc.F:L605–610) Explicit TKE budget update: production term

The production term `KappaM(i,j)*verticalShear(i,j)` is always non-negative, since both κ_m and S^2 are non-negative.

Typical values. In a wind-driven surface mixed layer with $\kappa_m \sim 0.01$ m^2 s^{-1} and $S^2 \sim 10^{-4}$ s^{-2}, the production term is $P \sim 10^{-6}$ m^2 s^{-3}. This is comparable to the dissipation term (see below), indicating a balance between shear production and dissipation in quasi-steady turbulence.

9 Boundary / Surface Layer Mixing

The mixing length ℓ is one of the most critical elements of the GGL90 scheme, determining the spatial scale over which turbulent eddies can transport momentum and scalars. Unlike some other turbulence closures that prescribe the mixing length purely from geometric considerations, GGL90 computes it from both turbulent energetics (TKE and buoyancy frequency) and geometric constraints imposed by boundaries and the water column structure.

The mixing length computation proceeds in two distinct stages:

1. **Initial estimate from Gaspar et al. (1990):** Compute an energetically-based mixing length $\ell^{(Gaspar)}$ that balances TKE with stratification (Section 9.1).

2. **Geometric constraints:** Apply boundary-aware constraints using one of three methods (Sections 9.2–9.4), controlled by runtime parameter `mxlMaxFlag`.

The final mixing length is then used to compute eddy viscosity and diffusivity (Section 10.8), and also appears in the TKE dissipation term (Section 10.5).

Historical context. The original Gaspar et al. (1990) paper proposed the energetic estimate, but left open the question of how to constrain the mixing length near boundaries. Blanke & Delecluse (1993) introduced a sophisticated two-way sweep algorithm (Method 2) that prevents unphysical penetration of mixing across stable layers, and this has become the de facto standard in many ocean models (see Appendix **??**).

Code organization. The mixing length computation is split between two subroutines:

- `GGL90_CALC` (lines 351–357): Computes the initial Gaspar estimate $\ell^{(Gaspar)}$.

- `GGL90_MIXINGLENGTH` (lines 7–421): Applies geometric constraints via one of three methods.

This section documents both stages in detail, with particular emphasis on Method 2, the most physically sophisticated approach.

9.1 Initial Estimate from Gaspar et al. (1990)

Before any geometric constraints are applied, GGL90 computes an initial mixing length estimate based purely on the local turbulent kinetic energy and stratification. This is the energetic mixing length from Gaspar et al. (1990), which represents the vertical scale over which a turbulent eddy with kinetic energy e can work against the ambient stratification.

9.1.1 Physical motivation

Consider a turbulent eddy with characteristic velocity scale $\sqrt{e}$ attempting to overturn a stratified water column with buoyancy frequency $N = \sqrt{N^2}$. The eddy can penetrate vertically until its kinetic energy is exhausted by work against buoyancy forces. Dimensional analysis gives:

$$\ell^{(Gaspar)} \sim \frac{\sqrt{e}}{N} \tag{20}$$

Gaspar et al. (1990) found that a factor of $\sqrt{2}$ improves agreement with observations and large-eddy simulations, yielding their equation (2.35):

$$\ell^{(Gaspar)} = \sqrt{2}\frac{\sqrt{e}}{\sqrt{\max(N^2, \varepsilon)}} \tag{21}$$

where ε is a small positive constant (typically 10^{-12} to 10^{-14} m^{-2} s^{-2}) to prevent division by zero in weakly stratified or unstratified regions.

Physical interpretation.

- In strongly stratified regions ($N^2 \gg \varepsilon$): $\ell^{(Gaspar)} \propto e/N^2$, so mixing is suppressed by stratification.
- In weakly stratified or convective regions ($N^2 \approx 0$ or $N^2 < 0$): The denominator is floored at $\sqrt{\varepsilon}$, allowing $\ell^{(Gaspar)}$ to grow in proportion to $\sqrt{e}$.
- At high TKE (e.g., near-surface wind-driven turbulence): $\ell^{(Gaspar)}$ increases, allowing deeper mixing.

9.1.2 Implementation in `GGL90_CALC`

The initial Gaspar mixing length is computed in the main driver routine `GGL90_CALC`, immediately after the buoyancy frequency N^2 has been diagnosed from the density field:

```
C     buoyancy frequency
          Nsquare(i,j,k) = gravity*gravitySign*recip_rhoConst
     &                   * sigmaR(i,j,k) * coordFac
C     vertical shear term (dU/dz)^2+(dV/dz)^2 is computed later
C     to save some memory
C     Initialise mixing length (eq. 2.35)
          GGL90mixingLength(i,j,k) = SQRTTWO *
     &         SQRTTKE(i,j,k)/SQRT( MAX(Nsquare(i,j,k),GGL90eps)
          )
     &         * mskLoc
         ENDDO
        ENDDO
       ENDDO
```

Listing 17: (ggl90_calc.F:L346–357) Initial mixing length estimate

Key code details:

- `SQRTTWO` is the constant $\sqrt{2} \approx 1.41421356$ (defined in `EEPARAMS.h`).
- `SQRTTKE(i,j,k)` is $\sqrt{e}$, precomputed earlier in the routine (lines 339–343) to avoid repeated square root operations.
- `GGL90eps` is the floor value ε for N^2, set by runtime parameter (default 10^{-12} m^{-2} s^{-2}).
- `mskLoc` is a local cell mask (`maskC(i,j,k)*maskC(i,j,k-1)`) that zeroes out the mixing length in dry cells or at dry interfaces.
- `coordFac` is a coordinate scaling factor (1 in z-coordinates, $g\rho_0$ in pressure coordinates) to convert between geometric and pressure vertical coordinates.

9.1.3 Coordinate transformation

Note that `Nsquare` is multiplied by `coordFac` to ensure dimensional consistency:

- In z-coordinates: `coordFac = 1`, so `Nsquare` has units of s^{-2}.
- In pressure coordinates: `coordFac = g*rhoConst`, so `Nsquare` is scaled appropriately for pressure-based vertical coordinates.

This ensures that the mixing length $\ell^{(Gaspar)}$ always has units of meters, regardless of the vertical coordinate system.

9.1.4 Typical values

For representative ocean conditions:

- **Surface mixed layer** (winter, deep convection): $e \sim 10^{-3}$ m^2 s^{-2}, $N^2 \sim 10^{-6}$ s^{-2} $\Rightarrow$ $\ell^{(Gaspar)} \sim 45$ m.
- **Thermocline** (strong stratification): $e \sim 10^{-5}$ m^2 s^{-2}, $N^2 \sim 10^{-4}$ s^{-2} $\Rightarrow$ $\ell^{(Gaspar)} \sim 0.45$ m.
- **Deep ocean** (weak stratification): $e \sim 10^{-6}$ m^2 s^{-2}, $N^2 \sim 10^{-7}$ s^{-2} $\Rightarrow$ $\ell^{(Gaspar)} \sim 4.5$ m.

These values are purely local and do not yet account for distance to boundaries, which is why the subsequent geometric constraint step is essential.

9.2 Method 0: Simple Distance to Boundaries

Method 0 (`mxlMaxFlag = 0`) is the simplest geometric constraint: cap the Gaspar mixing length at the total water column thickness, without any consideration of boundary proximity or vertical propagation of mixing constraints.

9.2.1 Algorithm

For each interior cell $k = 2, \ldots, N_r$:

$$\ell(i,j,k) = \min\left(\ell^{(Gaspar)}(i,j,k),\ H_{\text{col}}(i,j)\right) \tag{22}$$

where H_{col} is the total water column thickness:

$$H_{\text{col}}(i,j) = R_{\text{surf}}(i,j) - R_{\text{bottom}}(i,j) \tag{23}$$

Physical interpretation. This constraint simply prevents the mixing length from exceeding the total depth of the ocean at this horizontal location. It is a minimal sanity check, but does not capture the physically important constraint that mixing cannot extend beyond nearby boundaries (surface, bottom, or stable layers).

9.2.2 Implementation

The code for Method 0 is straightforward:

```
1       IF ( locMxlMaxFlag .EQ. 0 ) THEN
2
3        DO k=2,Nr
4         DO j=jMin,jMax
5          DO i=iMin,iMax
6 C      Use thickness of water column (inverse of recip_Rcol) as
    the
```

```
7  C       maximum length.
8                MaxLength =
9       &           ( Ro_surf(i,j,bi,bj) - R_low(i,j,bi,bj) )*
            recip_coordFac
10               GGL90mixingLength(i,j,k) = MIN(GGL90mixingLength(i,j,
                     k),
11      &                                       MaxLength)
12           ENDDO
13          ENDDO
14         ENDDO
```

Listing 18: (ggl90_mixinglength.F:L166–179) Method 0 constraint

Key variables:

- `Ro_surf(i,j,bi,bj)`: Position of the surface (z-coords) or top pressure (p-coords).
- `R_low(i,j,bi,bj)`: Position of the bottom (z-coords) or bottom pressure (p-coords).
- `recip_coordFac`: Inverse of coordinate scaling factor (1 in z-coords, $1/(g\rho_0)$ in p-coords).

9.2.3 When to use Method 0

Method 0 is rarely used in practice because it is too permissive: it allows unrealistically large mixing lengths in deep ocean columns, where turbulence should be confined to a thin layer near boundaries or shear zones. However, it can be useful for:

- **Debugging**: If you suspect that the more sophisticated methods (1 or 2) are causing numerical issues, Method 0 provides a minimal baseline.
- **Shallow water applications**: In very shallow domains (e.g., $H < 50$ m), the entire water column may be actively mixed, and Method 0 suffices.
- **Theoretical studies**: Method 0 isolates the energetic mixing length behavior without geometric constraints.

For realistic ocean simulations, Method 1 or (preferably) Method 2 should be used instead.

9.3 Method 1: One-Way Downward Sweep

Method 1 (`mxlMaxFlag = 1`) constrains the mixing length at each cell by the minimum distance to either the surface above or the bottom below. This is a significant improvement over Method 0, as it prevents mixing from penetrating beyond the nearest boundary.

9.3.1 Algorithm

For each interior cell $k = 2, \ldots, N_r$:

$$\ell(i,j,k) = \min\left(\ell^{(Gaspar)}(i,j,k),\ d_{\text{surf}}(i,j,k),\ d_{\text{bottom}}(i,j,k)\right) \tag{24}$$

where:

$$\begin{aligned} d_{\text{surf}}(i,j,k) &= R_{\text{surf}}(i,j) - r_F(k) \\ d_{\text{bottom}}(i,j,k) &= r_F(k) - R_{\text{bottom}}(i,j) \end{aligned} \tag{25}$$

and $r_F(k)$ is the vertical position of the lower face of cell k (i.e., the interface between cells k and $k+1$).

Physical interpretation. At each grid cell, the mixing length is limited by whichever boundary (top or bottom) is closer. For example:

- Near the surface: $d_{\text{surf}} < d_{\text{bottom}}$, so $\ell \leq d_{\text{surf}}$.
- Near the bottom: $d_{\text{bottom}} < d_{\text{surf}}$, so $\ell \leq d_{\text{bottom}}$.
- Mid-depth: The constraint depends on which boundary is closer.

This prevents the mixing length from extending into the "shadow" of a nearby boundary.

9.3.2 Implementation

```
1         ELSEIF ( locMxlMaxFlag .EQ. 1 ) THEN
2
3          DO k=2,Nr
4           DO j=jMin,jMax
5            DO i=iMin,iMax
6             MaxLength=MIN(Ro_surf(i,j,bi,bj)-rF(k),rF(k)-R_low(i,
                  j,bi,bj))
7       &           * recip_coordFac
8       c           MaxLength=MAX(MaxLength,20. _d 0)
9             GGL90mixingLength(i,j,k) = MIN(GGL90mixingLength(i,j,
                  k),
10      &                                    MaxLength)
11           ENDDO
12          ENDDO
13         ENDDO
```

Listing 19: (ggl90_mixinglength.F:L181–193) Method 1 constraint

Key details:

- `rF(k)`: Vertical position of the lower face of cell k (interface $k/k+1$).
- The commented-out line `MaxLength=MAX(MaxLength,20. _d 0)` would impose a lower bound of 20 m on the mixing length; this is disabled by default but could be useful in coarse-resolution models to prevent overly small mixing lengths.

9.3.3 Limitations

Method 1 is a substantial improvement over Method 0, but it has a critical limitation: **it does not account for vertical propagation of boundary constraints**. Consider a stratified water column with a shallow mixed layer of depth 50 m and a total column depth of 1000 m:

- At 30 m depth (within the mixed layer): $d_{\text{surf}} = 30$ m, $d_{\text{bottom}} = 970$ m $\Rightarrow \ell \leq 30$ m.
- At 70 m depth (just below the mixed layer base): $d_{\text{surf}} = 70$ m, $d_{\text{bottom}} = 930$ m $\Rightarrow \ell \leq 70$ m.

The problem: even though there is a strong pycnocline at 50 m depth that should block mixing, Method 1 allows ℓ to jump from 30 m to 70 m as we descend through the pycnocline. This violates the physical principle that mixing cannot propagate vertically through stable layers.

Method 2 (next subsection) addresses this limitation via a two-way sweep algorithm.

9.3.4 When to use Method 1

Method 1 is a reasonable choice for:

- **Weakly stratified domains**: If the water column is relatively homogeneous (e.g., polar regions in winter), the lack of vertical propagation is less problematic.
- **Coarse vertical resolution**: In models with $\Delta z > 50$ m, the distinction between Methods 1 and 2 may be small.
- **Computational cost**: Method 1 is slightly cheaper than Method 2 (single pass vs. double pass), though the difference is negligible in most applications.

For high-resolution simulations with realistic stratification, Method 2 is strongly preferred.

9.4 Method 2: Two-Way Sweep (Blanke & Delecluse 1993)

Method 2 (`mxlMaxFlag = 2`) is the most sophisticated and physically realistic approach, implementing the two-way sweep algorithm of Blanke & Delecluse (1993). This method ensures that the mixing length is constrained not only by distance to the nearest boundary, but also by the accumulated constraint from all cells above and below, preventing unphysical propagation of mixing across stable layers.

Recommended method. Method 2 is the standard choice for most modern ocean GCMs (e.g., Appendix **??**), as it produces the most realistic mixed layer depths and stratification profiles.

9.4.1 Physical motivation

The key insight of Blanke & Delecluse (1993) is that the mixing length at a given depth should be constrained by *all* the mixing lengths between that depth and the boundaries, not just the direct geometric distance to the boundaries. Specifically:

- If there is a thin layer with ℓ_{small} above the current cell, then the current cell's mixing length should not exceed $\ell_{\text{small}} + \Delta z$, where Δz is the thickness of one grid cell.
- Similarly, if there is a thin layer below with ℓ_{small}, the current cell should inherit this constraint.

This implements the physical constraint that mixing cannot "jump over" a stable layer with small mixing length.

9.4.2 Algorithm overview

Method 2 proceeds in three stages:

1. **Downward sweep**: Starting from the surface, propagate mixing length constraints downward:
$$\ell_{\text{down}}(k) = \min\left(\ell^{(Gaspar)}(k),\ \ell_{\text{down}}(k-1) + \Delta z_{k-1}\right) \tag{26}$$
2. **Upward sweep**: Starting from the bottom, propagate mixing length constraints upward:
$$\ell_{\text{up}}(k) = \min\left(\ell_{\text{up}}(k+1) + \Delta z_k,\ \ell_{\text{down}}(k)\right) \tag{27}$$
3. **Final constraint**: Take the minimum of the downward-swept and upward-swept values:
$$\ell(k) = \min\left(\ell_{\text{up}}(k),\ \ell_{\text{down}}(k)\right) \tag{28}$$

The downward sweep ensures that surface constraints propagate down, the upward sweep ensures that bottom constraints propagate up, and the final minimum ensures that the most restrictive constraint (from either direction) is applied.

9.4.3 Implementation in MITgcm

The code for Method 2 is more complex than Methods 0 or 1, and it must handle two different cases depending on whether the model uses z-coordinates or pressure coordinates. The vertical sweep direction differs between the two coordinate systems:

- **Z-coordinates**: Surface is at $k = 1$, bottom is at $k = N_r$. Downward sweep goes from $k = 2$ to N_r; upward sweep goes from $k = N_r - 1$ to 2.

- **Pressure coordinates**: Surface is at $k = N_r$, bottom is at $k = 1$. Downward sweep goes from $k = N_r - 1$ to 2; upward sweep goes from $k = 2$ to N_r.

We document the z-coordinate case here (lines 240–299), which is the most common. The p-coordinate case (lines 197–238) follows the same logic with reversed loop bounds.

Downward sweep (z-coordinates):

```
C      Downward sweep
         DO k=2,Nr
#ifdef ALLOW_AUTODIFF_TAMC
          kkey = k + (tkey-1)*Nr
CADJ STORE mxLength_Dn(:,:,k-1)
CADJ &     = comlev1_bibj_k, key = kkey, kind=isbyte
#endif /* ALLOW_AUTODIFF_TAMC */
          DO j=jMin,jMax
           DO i=iMin,iMax
            mxLength_Dn(i,j,k) = MIN(GGL90mixingLength(i,j,k),
     &           mxLength_Dn(i,j,k-1)+drF(k-1)*recip_coordFac)
           ENDDO
          ENDDO
         ENDDO
```

Listing 20: (ggl90_mixinglength.F:L245–258) Downward sweep in z-coordinates

Key details:

- `mxLength_Dn(i,j,k)` is the downward-swept mixing length, stored in a temporary array.
- The initialization `mxLength_Dn(i,j,1) = GGL90mixingLengthMin` (line 131) provides the surface boundary condition.
- `drF(k-1)` is the thickness of cell $k-1$, so `mxLength_Dn(i,j,k-1)+drF(k-1)` is the mixing length from the cell above plus the increment needed to reach cell k.

Upward sweep (z-coordinates):

```
C      Upward sweep, extra treatment of k=Nr for z-coordinates
C      because level Nr+1 is not available
         DO j=jMin,jMax
          DO i=iMin,iMax
           GGL90mixingLength(i,j,Nr) = MIN(GGL90mixingLength(i,j
               ,Nr),
     &          GGL90mixingLengthMin+drF(Nr)*recip_coordFac)
          ENDDO
         ENDDO
         DO k=Nr-1,2,-1
```

```
#ifdef ALLOW_AUTODIFF_TAMC
          kkey = k + (tkey-1)*Nr
C     It is important that the two k-levels of these fields are
     stored
C     in one statement because otherwise taf will only store
    one, which
C     is wrong (i.e. was wrong in previous versions).
CADJ STORE GGL90mixingLength(:,:,k+1), GGL90mixingLength(:,:,k)
CADJ &     = comlev1_bibj_k, key = kkey, kind=isbyte
#endif /* ALLOW_AUTODIFF_TAMC */
          DO j=jMin,jMax
           DO i=iMin,iMax
             GGL90mixingLength(i,j,k) = MIN(GGL90mixingLength(i,j
                 ,k),
     &            GGL90mixingLength(i,j,k+1)+drF(k)*
         recip_coordFac)
           ENDDO
          ENDDO
         ENDDO
```

Listing 21: (ggl90_mixinglength.F:L260–283) Upward sweep in z-coordinates

Key details:

- The upward sweep overwrites `GGL90mixingLength` directly (rather than using a temporary array), imposing the constraint from below.
- Special treatment at $k = N_r$ (bottom cell): Since there is no cell below, the constraint is simply `GGL90mixingLengthMin + drF(Nr)`, which allows at most a one-cell-thick mixing layer at the bottom.
- The upward sweep runs in reverse order ($k = N_r - 1$ down to 2).

Final constraint:

```
C     Impose minimum from downward sweep
       DO k=2,Nr
        DO j=jMin,jMax
         DO i=iMin,iMax
           GGL90mixingLength(i,j,k) = MIN(GGL90mixingLength(i,j,
               k),
     &                                    mxLength_Dn(i,j,k))
         ENDDO
        ENDDO
       ENDDO
```

Listing 22: (ggl90_mixinglength.F:L291–299) Final minimum of downward and upward sweeps

After the upward sweep, `GGL90mixingLength` contains the upward-swept value ℓ_{up}. This final loop takes the minimum with the downward-swept value ℓ_{down} stored in `mxLength_Dn`, yielding the final constrained mixing length.

9.4.4 Worked example: Idealized stratification

Consider an idealized water column with a shallow mixed layer and a sharp pycnocline:

Depth (m)	N^2 (s^{-2})	$\ell^{(Gaspar)}$ (m)	Δz (m)
10 (k=2)	10^{-6}	45	20
30 (k=3)	10^{-6}	45	20
50 (k=4)	10^{-4}	4.5	20
70 (k=5)	10^{-5}	14	20
90 (k=6)	10^{-5}	14	20

Downward sweep:

$$\begin{aligned}
&\ell_{\text{down}}(2) = \min(45, 0+0) = 0 \quad \text{(initialized at surface)}\\
&\ell_{\text{down}}(3) = \min(45, 0+20) = 20\\
&\ell_{\text{down}}(4) = \min(4.5, 20+20) = 4.5 \quad \text{(strong stratification limits mixing)}\\
&\ell_{\text{down}}(5) = \min(14, 4.5+20) = 14\\
&\ell_{\text{down}}(6) = \min(14, 14+20) = 14
\end{aligned}$$

Upward sweep: Starting from the bottom (assuming a bottom boundary condition $\ell_{\text{up}}(N_r) = \ell_{\min} + \Delta z$):

$$\begin{aligned}
&\ell_{\text{up}}(6) = \ell_{\text{down}}(6) = 14\\
&\ell_{\text{up}}(5) = \min(14+20, 14) = 14\\
&\ell_{\text{up}}(4) = \min(14+20, 4.5) = 4.5\\
&\ell_{\text{up}}(3) = \min(4.5+20, 20) = 20\\
&\ell_{\text{up}}(2) = \min(20+20, 0) = 0
\end{aligned}$$

Final mixing length:

$$\begin{aligned}
&\ell(2) = \min(\ell_{\text{up}}(2), \ell_{\text{down}}(2)) = \min(0, 0) = 0\\
&\ell(3) = \min(20, 20) = 20\\
&\ell(4) = \min(4.5, 4.5) = 4.5\\
&\ell(5) = \min(14, 14) = 14\\
&\ell(6) = \min(14, 14) = 14
\end{aligned}$$

Notice how the pycnocline at $k = 4$ (with $\ell^{(Gaspar)} = 4.5$ m) blocks the downward propagation of the large mixed-layer mixing lengths from $k = 2, 3$. The upward sweep ensures that this constraint also affects cells above the pycnocline.

9.4.5 Comparison of Methods 0, 1, and 2

Table 6 summarizes the three methods and their typical use cases.

Table 6: Comparison of GGL90 mixing length methods

Method	Flag	Algorithm	Best for
0	`mxlMaxFlag=0`	Cap at total column depth	Debugging, very shallow domains
1	`mxlMaxFlag=1`	Cap at distance to nearest boundary (surface or bottom)	Weakly stratified domains, coarse vertical resolution
2	`mxlMaxFlag=2`	Two-way sweep (Blanke & Delecluse 1993)	Realistic stratification, high vertical resolution, **recommended** (Appendix **??**)

Computational cost. Method 2 requires two vertical passes (downward + upward) plus a final min operation, making it roughly twice as expensive as Method 1. However, the mixing length computation is a small fraction of total GGL90 cost (dominated by the implicit TKE solve), so this difference is typically negligible (¡ 1% of total runtime).

Recommendation. Use Method 2 unless you have a specific reason to do otherwise.

9.5 Surface and Bottom Constraints (`mxlSurfFlag`)

In addition to the three main methods described above, the GGL90 scheme provides an optional surface constraint controlled by the runtime parameter `mxlSurfFlag`.

9.5.1 Purpose of `mxlSurfFlag`

In some configurations, the Gaspar mixing length (Eq. 21) may become very small near the surface due to weak TKE or strong stratification just below the surface. This can lead to unrealistically shallow mixed layers, particularly in coarse-resolution models where the first grid cell is thick (e.g., $\Delta z_1 > 10$ m).

Setting `mxlSurfFlag = .TRUE.` forces the mixing length at the surface interface (between cells 1 and 2) to be at least the thickness of the first grid cell:

$$\ell(k = 2) \geq \Delta z_1 \tag{29}$$

This ensures that the top two cells are always coupled by mixing, preventing the formation of an artificially thin "skin layer" at the surface.

9.5.2 Implementation

```
C-     Ensure mixing between first and second level
       IF (mxlSurfFlag) THEN
        DO j=jMin,jMax
         DO i=iMin,iMax
#ifdef ALLOW_SHELFICE
          IF ( useShelfIce ) THEN
           kSrf = MAX(1,kTopC(i,j,bi,bj))
           kTop = MIN(kSrf+1,Nr)
          ENDIF
#endif
          GGL90mixingLength(i,j,kTop)=drF(kSrf)*recip_coordFac
         ENDDO
        ENDDO
       ENDIF
```

Listing 23: (ggl90_mixinglength.F:L147–160) Surface constraint

Key details:

- `kSrf` is the surface level index (1 in z-coords, N_r in p-coords).
- `kTop` is the first interior interface index ($k = 2$ in z-coords, $k = N_r$ in p-coords).
- `drF(kSrf)` is the thickness of the surface cell.
- The `#ifdef ALLOW_SHELFICE` block adjusts `kSrf` and `kTop` for ice-shelf cavities, where the "surface" may be at a deeper level than $k = 1$.

This constraint is applied *before* Methods 0–2, so the subsequent geometric constraints can further reduce the mixing length if needed, but they cannot increase it below Δz_1 at $k = 2$.

9.5.3 When to use `mxlSurfFlag`

Enable `mxlSurfFlag = .TRUE.` when:

- The first grid cell is thick ($\Delta z_1 > 10$ m) and you observe unrealistically shallow mixed layers.
- You are using a coarse vertical resolution in the upper ocean (e.g., typical global configurations, see Appendix **??**).
- You want to ensure that wind-driven mixing always penetrates at least one grid cell below the surface.

Disable `mxlSurfFlag = .FALSE.` when:

- You have very high vertical resolution near the surface ($\Delta z_1 < 5$ m) and want to resolve thin diurnal warm layers or cool skin effects.
- You are performing sensitivity studies where the mixing length should be determined purely by the Gaspar formula and boundary constraints, without artificial floors.

Example application. Global ocean applications with coarse near-surface resolution (typically $\Delta z_1 = 10$ m) often use `mxlSurfFlag = .TRUE.` to ensure robust surface mixing (see Appendix **??** for a specific example).

9.5.4 Bottom boundary condition

At the bottom interface ($k = N_r$ in z-coords, $k = 1$ in p-coords), the mixing length is implicitly limited by the special treatment in the upward sweep (Method 2) or by the distance-to-bottom constraint (Method 1). No additional parameter analogous to `mxlSurfFlag` exists for the bottom boundary.

However, the parameter `GGL90mixingLengthMin` effectively sets a minimum mixing length everywhere, including at the bottom, to prevent numerical issues when $\ell \to 0$.

9.5.5 Regularization: `GGL90_REGULARIZE_MIXINGLENGTH`

An optional compile-time flag `GGL90_REGULARIZE_MIXINGLENGTH` modifies the final mixing length calculation to prevent abrupt transitions. When enabled, the code replaces:

$$\ell_{\text{final}} = \max(\ell, \ell_{\text{min}}) \tag{30}$$

with:

$$\ell_{\text{final}} = \sqrt{\ell^2 + \ell_{\text{min}}^2} \tag{31}$$

This provides a smoother transition as $\ell \to 0$, which can improve convergence in adjoint calculations (see Appendix **??**). The code appears in lines 390–416 of `ggl90_mixinglength.F`:

```
1          DO k=2,Nr
2           DO j=jMin,jMax
3            DO i=iMin,iMax
4 #ifdef GGL90_REGULARIZE_MIXINGLENGTH
5             MLtmp = SQRT( GGL90mixingLength(i,j,k)**2
6      &                  + GGL90mixingLengthMin**2 )
7 #else
8             MLtmp = MAX(GGL90mixingLength(i,j,k),
                  GGL90mixingLengthMin)
9 #endif
10            GGL90mixingLength(i,j,k) = MLtmp
11            rMixingLength(i,j,k) = 1. _d 0 / MLtmp
```

```
12                ENDDO
13              ENDDO
14            ENDDO
```

Listing 24: (ggl90_mixinglength.F:L402–415) Final mixing length with optional regularization

When to use regularization:

- Enable for adjoint/gradient-based optimization (see Appendix **??**), where smooth derivatives are important.
- Disable for forward-only simulations to maintain strict adherence to the published GGL90 formulation.

9.6 Langmuir Circulation Enhancement (Optional)

If the `ALLOW_GGL90_LANGMUIR` compile-time option is enabled, the mixing length can be further modified to account for Langmuir circulation effects (wind-wave interactions). This is an advanced feature documented separately in Section **??** and is *not* part of the standard GGL90 scheme.

The Langmuir enhancement computes a modified mixing length `LCmixingLength` that is larger than the standard `GGL90mixingLength` in the surface boundary layer, enhancing vertical mixing when wind and surface waves align. The details are beyond the scope of this section, but users should be aware that the mixing length values may differ from the standard GGL90 formulation when this option is active.

10 Core Diffusivity/Prognostic Update

This section describes the complete time-integration procedure for TKE and the subsequent computation of eddy viscosity and diffusivity coefficients.

10.1 TKE Evolution and Budget

The turbulent kinetic energy (TKE) e is the sole prognostic variable in the GGL90 scheme. Its evolution is governed by a budget equation that balances production (from shear and surface forcing), buoyancy effects (destruction by stable stratification, or enhancement by convection), dissipation (conversion to heat at small scales), and diffusive transport. This section documents the TKE equation as implemented in MITgcm, including the time-splitting between explicit and implicit terms.

10.2 Governing Equation

The TKE evolution equation implemented in GGL90 is:

$$\frac{\partial e}{\partial t} = \underbrace{\kappa_m S^2}_{\text{shear production}} - \underbrace{\kappa_h N^2}_{\text{buoyancy}} - \underbrace{c_\varepsilon \frac{e^{3/2}}{\ell}}_{\text{dissipation}} + \underbrace{\frac{\partial}{\partial z}\left(\kappa_e \frac{\partial e}{\partial z}\right)}_{\text{vertical diffusion}} + \underbrace{F_{\text{ext}}}_{\text{external sources}} \tag{32}$$

where:

- e: TKE per unit mass ($\text{m}^2\ \text{s}^{-2}$)
- κ_m: Eddy viscosity ($\text{m}^2\ \text{s}^{-1}$), computed as $\kappa_m = c_\mu \ell \sqrt{e}$ (Section 10.8)
- S^2: Squared vertical shear of horizontal velocity, $S^2 = (\partial u/\partial z)^2 + (\partial v/\partial z)^2$ (s^{-2})
- κ_h: Eddy diffusivity for scalars ($\text{m}^2\ \text{s}^{-1}$), related to κ_m via the turbulent Prandtl number (Section 10.8)
- N^2: Squared buoyancy frequency (s^{-2}), $N^2 = -(g/\rho_0)\,\partial\rho/\partial z$
- c_ε: Dissipation constant (dimensionless), typically 0.7
- ℓ: Mixing length (m), computed by `GGL90_MIXINGLENGTH` (Section 9.4)
- κ_e: Diffusivity of TKE itself ($\text{m}^2\ \text{s}^{-1}$), $\kappa_e = \alpha_e \kappa_m$ with $\alpha_e \approx 1$
- F_{ext}: External TKE sources (optional), including IDEMIX internal wave dissipation (Section 11.1) and Langmuir circulation (Section **??**)

Physical interpretation of each term:

1. **Shear production ($\kappa_m S^2$):** Extraction of kinetic energy from the mean flow by turbulent Reynolds stresses. Always positive (source term).
2. **Buoyancy ($-\kappa_h N^2$):**
   - If $N^2 > 0$ (stable stratification): Buoyancy work is negative (sink term), as turbulent eddies must do work against gravity to mix denser fluid upward.
   - If $N^2 < 0$ (convective instability): Buoyancy work is positive (source term), as potential energy is released by gravitational overturning.
3. **Dissipation ($c_\varepsilon e^{3/2}/\ell$):** Irreversible conversion of TKE to heat via molecular viscosity at the smallest turbulent scales. Always positive (sink term). Note that the dissipation rate scales as $e^{3/2}$ (Kolmogorov's $-5/3$ law) and is inversely proportional to the mixing length ℓ.
4. **Vertical diffusion:** Redistribution of TKE in the vertical by turbulent transport. This is typically a down-gradient flux (from high TKE to low TKE regions).
5. **External sources (F_{ext}):** Additional TKE sources from parameterized processes not explicitly resolved in the shear production term. Examples include internal wave breaking (IDEMIX) and Langmuir circulation.

Coordinate-invariant form. The equation above is written in z-coordinates (depth increasing downward, z positive upward). In pressure coordinates, vertical derivatives $\partial/\partial z$ are replaced by $\partial/\partial p$ with appropriate scaling factors (`coordFac` in the code).

Boundary conditions. Equation (32) is subject to:

- **Surface**: Dirichlet condition $e = B_0 u_*^2$ at $z = 0$, where u_* is the friction velocity and $B_0 \approx 3.75$ (Section 7.1).
- **Bottom**: Either Dirichlet ($e = B_0 u_{*,\text{bottom}}^2$) or Neumann ($\partial e/\partial z = 0$), controlled by `GGL90_dirichlet` (Section **??**).

10.3 Production Term

The shear production term $P = \kappa_m S^2$ represents the extraction of kinetic energy from the mean flow by turbulent Reynolds stresses.

10.3.1 Vertical shear computation

The squared vertical shear S^2 is computed in `GGL90_CALC` immediately after the mixing length calculation. MITgcm provides two options for computing S^2, controlled by the runtime parameter `calcMeanVertShear`:

Option 1: Average of four components (`calcMeanVertShear = .TRUE.`) Compute the shear at each of the four surrounding velocity points (two U-points, two V-points), then average:

$$S^2(i,j,k) = \frac{1}{2}\left[\left(\frac{\Delta u_1}{\Delta z}\right)^2 + \left(\frac{\Delta u_2}{\Delta z}\right)^2 + \left(\frac{\Delta v_1}{\Delta z}\right)^2 + \left(\frac{\Delta v_2}{\Delta z}\right)^2\right] \tag{33}$$

where $\Delta u_1 = u(i,j,k-1) - u(i,j,k)$, $\Delta u_2 = u(i+1,j,k-1) - u(i+1,j,k)$, and similarly for v.

```
1         IF ( calcMeanVertShear ) THEN
2 C     by averaging (@ grid-cell center) the 4 vertical shear
    compon @ U,V pos.
3         DO j=jMin,jMax
4          DO i=iMin,iMax
5           tempU  = ( uVel( i  ,j,km1,bi,bj) - uVel( i  ,j,k,bi,bj
              ) )
6           tempUp = ( uVel(i+1,j,km1,bi,bj) - uVel(i+1,j,k,bi,bj
              ) )
7           tempV  = ( vVel(i, j  ,km1,bi,bj) - vVel(i, j  ,k,bi,bj
              ) )
8           tempVp = ( vVel(i,j+1,km1,bi,bj) - vVel(i,j+1,k,bi,bj
              ) )
```

```
            verticalShear(i,j) = (
     &                    ( tempU*tempU + tempUp*tempUp )
     &                  + ( tempV*tempV + tempVp*tempVp )
     &                          )*halfRL*recip_drC(k)*recip_drC(
        k)
     &                           *coordFac*coordFac
          ENDDO
         ENDDO
```

Listing 25: (ggl90_calc.F:L468–482) Vertical shear: Option 1

Option 2: Shear of averaged flow (`calcMeanVertShear = .FALSE.`) First average the velocity components to the cell center, then compute shear:

$$S^2(i,j,k) = \left(\frac{\bar{u}(k-1) - \bar{u}(k)}{\Delta z}\right)^2 + \left(\frac{\bar{v}(k-1) - \bar{v}(k)}{\Delta z}\right)^2 \tag{34}$$

where $\bar{u}(k) = \frac{1}{2}[u(i,j,k) + u(i+1,j,k)]$ and $\bar{v}(k) = \frac{1}{2}[v(i,j,k) + v(i,j+1,k)]$.

```
        ELSE
C     from the averaged flow at grid-cell center (2 compon x 2
    pos.)
         DO j=jMin,jMax
          DO i=iMin,iMax
           tempU = ( ( uVel(i,j,km1,bi,bj) + uVel(i+1,j,km1,bi,
               bj) )
     &             -( uVel(i,j,k  ,bi,bj) + uVel(i+1,j,k  ,bi,
        bj) )
     &           )*halfRL*recip_drC(k)
     &            *coordFac
           tempV = ( ( vVel(i,j,km1,bi,bj) + vVel(i,j+1,km1,bi,
               bj) )
     &             -( vVel(i,j,k  ,bi,bj) + vVel(i,j+1,k  ,bi,
        bj) )
     &           )*halfRL*recip_drC(k)
     &            *coordFac
           verticalShear(i,j) = tempU*tempU + tempV*tempV
          ENDDO
         ENDDO
        ENDIF
```

Listing 26: (ggl90_calc.F:L483–498) Vertical shear: Option 2

Which option to use? Option 1 is more accurate on the Arakawa C-grid, as it avoids spurious cancellation of checkerboard modes. Option 2 is slightly cheaper computationally.

The difference is typically small in well-resolved simulations. Global applications typically use Option 1 (`calcMeanVertShear = .TRUE.`, see Appendix ??).

10.3.2 Production term calculation

Once S^2 and κ_m are known, the production term is straightforward:

```
1         GGL90TKE(i,j,k,bi,bj) = GGL90TKE(i,j,k,bi,bj)
2      &          + deltaTloc*(
3      &          + KappaM(i,j)*verticalShear(i,j)
4      &          - KappaH*Nsquare(i,j,k)
5      &          - TKEdissipation
6      &          )
```

Listing 27: (ggl90_calc.F:L605–610) Explicit TKE budget update: production term

The production term `KappaM(i,j)*verticalShear(i,j)` is always non-negative, since both κ_m and S^2 are non-negative.

Typical values. In a wind-driven surface mixed layer with $\kappa_m \sim 0.01$ m^2 s^{-1} and $S^2 \sim 10^{-4}$ s^{-2}, the production term is $P \sim 10^{-6}$ m^2 s^{-3}. This is comparable to the dissipation term (see below), indicating a balance between shear production and dissipation in quasi-steady turbulence.

10.4 Buoyancy Term

The buoyancy term $B = -\kappa_h N^2$ represents the work done by buoyancy forces on turbulent eddies.

10.4.1 Sign convention and physical interpretation

The sign of the buoyancy term depends on the stratification:

- **Stable stratification ($N^2 > 0$):** The term $-\kappa_h N^2 < 0$ is a sink of TKE. Turbulent eddies must work against buoyancy to lift heavy fluid (or depress light fluid), converting kinetic energy to potential energy. This suppresses turbulence.
- **Neutral stratification ($N^2 \approx 0$):** The buoyancy term is negligible, and TKE evolution is controlled by shear production and dissipation.
- **Unstable stratification ($N^2 < 0$):** The term $-\kappa_h N^2 > 0$ is a source of TKE. Convective overturning releases potential energy, which is converted to kinetic energy. This enhances turbulence.

Relation to the Richardson number. The ratio of buoyancy to production defines the flux Richardson number:

$$Ri_f = \frac{-\kappa_h N^2}{\kappa_m S^2} = -\frac{N^2}{Pr_t\, S^2} \tag{35}$$

where $Pr_t = \kappa_m/\kappa_h$ is the turbulent Prandtl number (Section 10.10). A critical value $Ri_f \approx 0.2$ separates turbulent ($Ri_f < 0.2$) from laminar ($Ri_f > 0.2$) regimes.

10.4.2 Implementation

The buoyancy term is computed alongside the production term in the explicit TKE update:

```
1          GGL90TKE(i,j,k,bi,bj) = GGL90TKE(i,j,k,bi,bj)
2       &        + deltaTloc*(
3       &        + KappaM(i,j)*verticalShear(i,j)
4       &        - KappaH*Nsquare(i,j,k)
5       &        - TKEdissipation
6       &        )
```

Listing 28: (ggl90_calc.F:L605–610) Explicit TKE budget update: buoyancy term

The term `- KappaH*Nsquare(i,j,k)` directly implements $-\kappa_h N^2$.

Coordinate scaling. Recall that `Nsquare` is pre-multiplied by `coordFac` (line 347–348 of `ggl90_calc.F`) to ensure dimensional consistency in pressure coordinates. This is automatically accounted for in the buoyancy term.

10.5 Dissipation Term

The dissipation term $\varepsilon = c_\varepsilon e^{3/2}/\ell$ represents the irreversible conversion of TKE to heat at the smallest scales of turbulence (the Kolmogorov scale).

10.5.1 Physical basis

In fully developed turbulence, the TKE dissipation rate ε scales as:

$$\varepsilon \sim \frac{e^{3/2}}{\ell} \tag{36}$$

This scaling arises from dimensional analysis: ε has units of $\mathrm{m^2\ s^{-3}}$, so the only combination of TKE (e, units $\mathrm{m^2\ s^{-2}}$) and length scale (ℓ, units m) that yields the correct dimensions is $e^{3/2}/\ell$.

The proportionality constant c_ε is an empirical parameter, typically set to 0.7 based on atmospheric boundary layer observations and large-eddy simulations. In MITgcm, c_ε is the runtime parameter `GGL90ceps`.

10.5.2 Implementation: Semi-implicit time integration

The dissipation term is unique among the TKE budget terms in that it is treated *semi-implicitly*: a fraction is computed explicitly, and the remainder is treated implicitly to ensure numerical stability.

Explicit part:

```
C     dissipation term
         TKEdissipation = explDissFac*GGL90ceps
     &        *SQRTTKE(i,j,k)*rMixingLength(i,j,k)
     &        *GGL90TKE(i,j,k,bi,bj)
C     partial update with sum of explicit contributions
         GGL90TKE(i,j,k,bi,bj) = GGL90TKE(i,j,k,bi,bj)
     &        + deltaTloc*(
     &        + KappaM(i,j)*verticalShear(i,j)
     &        - KappaH*Nsquare(i,j,k)
     &        - TKEdissipation
     &        )
```

Listing 29: (ggl90_calc.F:L600–610) Explicit dissipation term

Here, `explDissFac` is the explicit fraction of the dissipation term. The formula for `TKEdissipation` is:

$$\varepsilon_{\text{explicit}} = f_{\text{expl}}\, c_\varepsilon \frac{\sqrt{e}\cdot e}{\ell} = f_{\text{expl}}\, c_\varepsilon \frac{e^{3/2}}{\ell} \tag{37}$$

where f_{expl} = `explDissFac` and `rMixingLength = 1/ell` is precomputed to avoid repeated divisions.

Implicit part: The implicit fraction $(1 - f_{\text{expl}})$ appears in the center diagonal of the tridiagonal matrix used for the implicit TKE solve:

```
         b3d(i,j,k) = 1. _d 0 - c3d(i,j,k) - a3d(i,j,k)
     &        + implDissFac*deltaTloc*GGL90ceps*SQRTTKE(i,j,k)
     &        * rMixingLength(i,j,k)
     &        * maskC(i,j,k,bi,bj)*maskC(i,j,km1,bi,bj)
```

Listing 30: (ggl90_calc.F:L746–749) Implicit dissipation in matrix diagonal

The term `+ implDissFac*deltaTloc*GGL90ceps*SQRTTKE(i,j,k)*rMixingLength(i,j,k)` adds the implicit dissipation contribution to the diagonal of the matrix.

10.5.3 Explicit/implicit splitting

The fractions `explDissFac` and `implDissFac` are set in lines 226–246 of `ggl90_calc.F`:

```
C     Explicit/implicit TKE dissipation
      explDissFac = 0. _d 0
```

```
3         implDissFac = 1. _d 0
4         IF ( GGL90dumpFreq.LT.dTtracerLev(1) ) THEN
5 C        Recover the old explicit-only-formulation by
6 C        setting explDissFac=1 and implDissFac=0
7          explDissFac = 1. _d 0
8          implDissFac = 0. _d 0
9         ENDIF
10 C      The implicit part is not coded very efficiently in terms
11 C      of TAF dependencies (with the CG3D routine called by the
12 C      GGL90_DO_EXCH macro), so we default to DIVA_EXCH instead,
13 C      and still use explDissFac=0 and implDissFac=1
14 #ifdef ALLOW_AUTODIFF_TAMC
15        IF ( useGGL90inAdMode ) THEN
16         explDissFac = 0. _d 0
17         implDissFac = 1. _d 0
18        ENDIF
19 #endif /* ALLOW_AUTODIFF_TAMC */
```

Listing 31: (ggl90_calc.F:L226–246) Explicit/implicit dissipation fractions

Default behavior:

- **Standard forward run**: `explDissFac = 0`, `implDissFac = 1` (fully implicit dissipation). This is unconditionally stable for any time step.
- **Adjoint mode**: Same as forward (fully implicit).
- **Legacy mode**: If `GGL90dumpFreq < dTtracerLev(1)` (which is never true in practice), revert to the old fully explicit scheme (`explDissFac = 1`, `implDissFac = 0`). This is retained for backward compatibility but is **not recommended** due to potential instability.

Why semi-implicit? The dissipation term is nonlinear in e (proportional to $e^{3/2}$), which can cause CFL-like instabilities if treated purely explicitly with large time steps. Implicit treatment linearizes the term about the current time step ($\partial\varepsilon/\partial e \approx 3/2 \cdot \varepsilon/e$), ensuring stability without requiring prohibitively small time steps.

10.6 Vertical Diffusion of TKE

The vertical diffusion term $\partial/\partial z(\kappa_e\, \partial e/\partial z)$ represents turbulent transport of TKE itself, spreading TKE from high-energy regions (e.g., near the surface) to low-energy regions (e.g., the thermocline).

10.6.1 Diffusivity of TKE: κ_e

The diffusivity of TKE is related to the eddy viscosity κ_m via the constant α_e:

$$\kappa_e = \alpha_e \, \kappa_m \tag{38}$$

In GGL90, α_e = `GGL90alpha` is a runtime parameter, typically set to 1.0 (following Gaspar et al. 1990).

The code computes κ_e in lines 593–598 of `ggl90_calc.F`:

```
1         DO j=jMin,jMax
2          DO i=iMin,iMax
3 C          diffusivity
4            KappaH = KappaM(i,j)/TKEPrandtlNumber(i,j,k)
5            KappaE(i,j,k) = GGL90alpha * KappaM(i,j)
6       &          * maskC(i,j,k,bi,bj)*maskC(i,j,k-1,bi,bj)
```

Listing 32: (ggl90_calc.F:L593–598) Diffusivity of TKE

Note that κ_e is stored at cell interfaces (W-points), just like κ_m and κ_h.

10.6.2 Implicit vertical diffusion solver

The vertical diffusion of TKE is treated *fully implicitly* using a tridiagonal solve. This is necessary for numerical stability, as explicit diffusion would impose a restrictive CFL condition:

$$\Delta t \leq \frac{\Delta z^2}{2\kappa_e} \tag{39}$$

which would be prohibitively small in regions with large κ_e (e.g., near the surface under strong wind forcing).

Tridiagonal matrix construction. The implicit diffusion equation is discretized as:

$$\frac{e_k^{n+1} - e_k^*}{\Delta t} = \frac{1}{\Delta z_k}\left[\kappa_{e,k-1/2}\frac{e_{k-1}^{n+1} - e_k^{n+1}}{\Delta z_{k-1/2}} - \kappa_{e,k+1/2}\frac{e_k^{n+1} - e_{k+1}^{n+1}}{\Delta z_{k+1/2}}\right] \tag{40}$$

where e_k^* is the TKE after the explicit update (production, buoyancy, dissipation), and e_k^{n+1} is the final TKE after implicit diffusion. Rearranging yields a tridiagonal system:

$$a_k e_{k-1}^{n+1} + b_k e_k^{n+1} + c_k e_{k+1}^{n+1} = e_k^* \tag{41}$$

The coefficients a_k, b_k, c_k are computed in lines 668–760 of `ggl90_calc.F`:

```
1 C     ==========================================================
2 C     Implicit time step to update TKE for k=1,Nr;
3 C     TKE(Nr+1)=0 by default;
4 C     for pressure coordinates, this translates into
5 C     TKE(1)   = 0, TKE(Nr+1) is the surface value
```

```
C     ==========================================================
C     set up matrix
C---  Lower diagonal
      DO j=jMin,jMax
       DO i=iMin,iMax
         a3d(i,j,1) = 0. _d 0
       ENDDO
      ENDDO
      DO k=2,Nr
       km1=MAX(2,k-1)
       DO j=jMin,jMax
        DO i=iMin,iMax
         IF ( usingPCoords) km1=MIN(Nr,MAX(kSurfC(i,j,bi,bj)+1,
           k-1))
C-    We keep recip_hFacC in the diffusive flux calculation,
C-    but no hFacC in TKE volume control
C-    No need for maskC(k-1) with recip_hFacC(k-1)
         a3d(i,j,k) = -deltaTloc
     &        *recip_drF(k-1)*recip_hFacC(i,j,k-1,bi,bj)
     &        *.5 _d 0*(KappaE(i,j, k )+KappaE(i,j,km1))
     &        *recip_drC(k)*maskC(i,j,k,bi,bj)*recip_hFacI(i,j,
        k)
     &        *coordFac*coordFac
        ENDDO
       ENDDO
      ENDDO
C---  Upper diagonal
      DO j=jMin,jMax
       DO i=iMin,iMax
         c3d(i,j,1)  = 0. _d 0
       ENDDO
      ENDDO
      DO k=2,Nr
       kp1=MIN(k+1,Nr)
       DO j=jMin,jMax
        DO i=iMin,iMax
         IF ( usingZCoords ) kp1=MAX(1,MIN(klowC(i,j,bi,bj),k
           +1))
C-    We keep recip_hFacC in the diffusive flux calculation,
C-    but no hFacC in TKE volume control
C-    No need for maskC(k) with recip_hFacC(k)
         c3d(i,j,k) = -deltaTloc
     &        *recip_drF( k ) * recip_hFacC(i,j,k,bi,bj)
     &        *.5 _d 0*(KappaE(i,j,k)+KappaE(i,j,kp1))
     &        *recip_drC(k)*maskC(i,j,k-1,bi,bj)*recip_hFacI(i,
```

```
          j,k)
48      &          *coordFac*coordFac
49            ENDDO
50          ENDDO
51        ENDDO
```

Listing 33: (ggl90_calc.F:L668–720) Tridiagonal matrix setup for implicit TKE diffusion

Key implementation details:

- `a3d(i,j,k)` is the lower diagonal coefficient (coupling to cell $k-1$).
- `c3d(i,j,k)` is the upper diagonal coefficient (coupling to cell $k+1$).
- `b3d(i,j,k)` is the center diagonal coefficient (cell k), computed as $b_k = 1 - a_k - c_k +$ (implicit dissipation).
- The factor `0.5*(KappaE(i,j,k)+KappaE(i,j,km1))` is the average of κ_e at the two adjacent interfaces, which is appropriate for a cell-centered TKE variable with interface-located diffusivities.
- `coordFac*coordFac` accounts for coordinate transformation (z vs. p coords).

10.6.3 Tridiagonal solve

After the matrix is constructed, the system is solved by a call to the generic MITgcm tridiagonal solver `SOLVE_TRIDIAGONAL`:

```
1       CALL SOLVE_TRIDIAGONAL( iMin, iMax, jMin, jMax,
2      I                        a3d, b3d, c3d,
3      U                        GGL90TKE(1-OLx,1-OLy,1,bi,bj),
4      I                        errCode, bi, bj, myThid )
```

Listing 34: (ggl90_calc.F:L900–903) Solve tridiagonal system for TKE

On entry, `GGL90TKE` contains the right-hand side e^* (TKE after explicit update). On exit, it contains the solution e^{n+1} (TKE after implicit diffusion and dissipation).

10.7 Time Integration Scheme: Operator Splitting

The GGL90 TKE equation is integrated using a **split-explicit-implicit** scheme:

1. **Explicit step**: Update TKE with production, buoyancy, explicit fraction of dissipation, and any external sources (IDEMIX, Langmuir, horizontal diffusion):

$$e^* = e^n + \Delta t \left(\kappa_m S^2 - \kappa_h N^2 - f_{\text{expl}} \frac{c_\varepsilon e^{3/2}}{\ell} + F_{\text{ext}} \right) \tag{42}$$

2. **Implicit step**: Solve for e^{n+1} including implicit dissipation and vertical diffusion:

$$e^{n+1} = \mathcal{L}^{-1}\left[e^{*} - (1 - f_{\text{expl}})\frac{c_{\varepsilon}(e^{n+1})^{3/2}}{\ell}\right] \tag{43}$$

where $\mathcal{L}^{-1}$ represents the tridiagonal solve for vertical diffusion.

This splitting is unconditionally stable for any time step Δt, provided that the explicit dissipation fraction f_{expl} is small (default: $f_{\text{expl}} = 0$).

10.7.1 Time step selection

The TKE time step `deltaTloc` is set equal to the tracer time step `dTtracerLev(1)` (line 221 of `ggl90_calc.F`):

```
1       deltaTloc = dTtracerLev(1)
```

Listing 35: (ggl90_calc.F:L221) TKE time step

This ensures that TKE evolves synchronously with temperature and salinity, which is physically appropriate since turbulent mixing of TKE and scalars occurs on the same time scales.

Sub-cycling TKE (optional). For very high Reynolds number flows or very coarse time stepping, it may be beneficial to sub-cycle the TKE equation (i.e., take multiple TKE steps per tracer step). However, this is not currently implemented in the standard GGL90 package and would require code modifications.

10.7.2 Horizontal diffusion of TKE (optional)

If the compile-time flag `ALLOW_GGL90_HORIZDIFF` is enabled and `GGL90diffTKEh > 0`, the code also includes horizontal diffusion of TKE:

$$\frac{\partial e}{\partial t} = \ldots + \nabla_h \cdot (\kappa_{h,\text{TKE}} \nabla_h e) \tag{44}$$

This is computed explicitly (lines 381–434 of `ggl90_calc.F`) and added to the TKE budget before the implicit solve (lines 638–648):

```
1  #ifdef ALLOW_GGL90_HORIZDIFF
2         IF ( GGL90diffTKEh .GT. 0. _d 0 ) THEN
3  C--     Add horiz. diffusion tendency
4          DO j=jMin,jMax
5           DO i=iMin,iMax
6            GGL90TKE(i,j,k,bi,bj) = GGL90TKE(i,j,k,bi,bj)
7       &                            + gTKE(i,j)*deltaTloc
8           ENDDO
9          ENDDO
10        ENDIF
```

```
#endif /* ALLOW_GGL90_HORIZDIFF */
```

Listing 36: (ggl90_calc.F:L638–648) Horizontal diffusion of TKE

When to use horizontal diffusion. Horizontal TKE diffusion is rarely used in realistic ocean simulations, as it is not physically motivated (TKE should be generated locally by shear and buoyancy, not transported horizontally). However, it can be useful for:

- Smoothing spurious TKE gradients in coarse-resolution models.
- Numerical stability in eddy-permitting or eddy-resolving configurations with very strong mesoscale variability.

The default is `GGL90diffTKEh = 0` (no horizontal diffusion).

10.8 Eddy Coefficient Calculation

The GGL90 scheme diagnoses eddy viscosity κ_m and eddy diffusivity κ_h from the prognostic TKE field e and the mixing length ℓ. These coefficients are then used by the main model to compute vertical fluxes of momentum, temperature, and salinity.

The calculation proceeds in three steps:

1. Compute eddy viscosity $\kappa_m = c_\mu \ell \sqrt{e}$ (Section 10.9).
2. Compute TKE Prandtl number $Pr_t = f(Ri)$ from the local Richardson number (Section 10.10).
3. Compute eddy diffusivity $\kappa_h = \kappa_m / Pr_t$ (Section 10.11).
4. Apply background floors and maximum caps (Section 10.12).

These coefficients are computed at every interior interface (k=2 to Nr) and then transferred to the main model via `GGL90_CALC_VISC` and `GGL90_CALC_DIFF` (Section 12).

10.9 Eddy Viscosity (κ_m)

10.9.1 Theoretical basis

The eddy viscosity in GGL90 is based on Prandtl's mixing length hypothesis combined with the TKE energy scale:

$$\kappa_m = c_\mu \, \ell \, \sqrt{e} \tag{45}$$

where:

- c_μ (`GGL90ck`): Dimensionless constant, typically $c_\mu = 0.1$ (Gaspar et al. 1990).
- ℓ (`GGL90mixingLength`): Mixing length (m), computed by `GGL90_MIXINGLENGTH` (Section 9.4).
- $\sqrt{e}$ (`SQRTTKE`): Characteristic turbulent velocity scale (m s^{-1}).

Physical interpretation. The product $\ell\sqrt{e}$ has dimensions of $\mathrm{m}^2\ \mathrm{s}^{-1}$ (kinematic viscosity). It represents the rate at which turbulent eddies of size ℓ and velocity $\sqrt{e}$ transport momentum vertically. The constant c_μ is an empirical factor determined from observations and large-eddy simulations.

Comparison to K-profile parameterization (KPP). In KPP, the eddy viscosity is prescribed as a shape function times a boundary layer depth scale. In GGL90, κ_m emerges dynamically from the TKE and mixing length, which respond to local stratification and shear. This makes GGL90 more physically consistent, but also more computationally expensive due to the prognostic TKE equation.

10.9.2 Implementation

The eddy viscosity is computed in lines 436–465 of `ggl90_calc.F`:

```
1          DO j=jMin,jMax
2           DO i=iMin,iMax
3            KappaM(i,j) = GGL90ck*GGL90mixingLength(i,j,k)*
                SQRTTKE(i,j,k)
4           ENDDO
5          ENDDO
6         DO j=jMin,jMax
7          DO i=iMin,iMax
8           GGL90visctmp(i,j,k) = MAX( KappaM(i,j),diffKrNrS(k)
9       &                                 * recip_coordFac*
            recip_coordFac )
10      &         * maskC(i,j,k,bi,bj)*maskC(i,j,k-1,bi,bj)
11 C         note: storing GGL90visctmp like this, and using it
    later to compute
12 C               GGL9rdiffKr etc. is robust in case of smoothing
    (e.g. see OPA)
13          KappaM(i,j) = MAX( KappaM(i,j),viscArNr(k)
14      &                      * recip_coordFac*recip_coordFac )
15      &                      * maskC(i,j,k,bi,bj)*maskC(i,j,k-1,bi
            ,bj)
16          ENDDO
17         ENDDO
```

Listing 37: (ggl90_calc.F:L446–465) Eddy viscosity calculation

Key implementation details:

- Line 448: κ_m is computed from Eq. (45).
- Lines 456–458: `GGL90visctmp` is a copy of κ_m (before applying the viscosity floor), floored at the background scalar diffusivity `diffKrNrS(k)`. This is used later to com-

pute diffusivities for temperature and salinity, ensuring that scalar diffusion is never less than the background value.

- Lines 461–463: The actual `KappaM` used for momentum is floored at the background viscosity `viscArNr(k)`, which is typically larger than `diffKrNrS(k)` (see Section 10.12).
- `maskC(i,j,k)*maskC(i,j,k-1)`: Ensures that κ_m is zero at dry interfaces (e.g., where one of the adjacent cells is land or ice).
- `recip_coordFac*recip_coordFac`: Coordinate transformation factor for pressure coordinates. In z-coordinates, `coordFac = 1`, so this factor is unity.

10.9.3 Typical values

For representative ocean conditions:

- **Surface mixed layer** (wind-driven turbulence): $\ell \sim 20$ m, $\sqrt{e} \sim 0.03$ m s^{-1}, $c_\mu = 0.1$ $\Rightarrow \kappa_m \sim 0.06$ m^2 s^{-1}.
- **Thermocline** (strong stratification): $\ell \sim 1$ m, $\sqrt{e} \sim 0.003$ m s^{-1}, $c_\mu = 0.1$ $\Rightarrow \kappa_m \sim 3 \times 10^{-4}$ m^2 s^{-1}.
- **Deep ocean** (weak background turbulence): $\ell \sim 5$ m, $\sqrt{e} \sim 0.001$ m s^{-1}, $c_\mu = 0.1$ $\Rightarrow \kappa_m \sim 5 \times 10^{-4}$ m^2 s^{-1}.

These values span 2–3 orders of magnitude, illustrating the highly variable nature of ocean turbulence.

10.10 TKE Prandtl Number

The turbulent Prandtl number Pr_t relates the eddy viscosity κ_m to the eddy diffusivity for scalars κ_h:

$$Pr_t = \frac{\kappa_m}{\kappa_h} \tag{46}$$

In a molecularly-dominated flow, the Prandtl number is determined by fluid properties (e.g., $Pr \approx 7$ for seawater at 20°C). In turbulent flow, Pr_t depends on the flow stability, quantified by the Richardson number.

10.10.1 Formulation as a function of Richardson number

GGL90 uses an empirical relationship between Pr_t and the local gradient Richardson number Ri:

$$Ri = \frac{N^2}{S^2} = \frac{-(g/\rho_0)\,\partial\rho/\partial z}{(\partial u/\partial z)^2 + (\partial v/\partial z)^2} \tag{47}$$

The Prandtl number formulation is:

$$Pr_t = \begin{cases} 1 & \text{if } Ri < 0.2 \\ 5\,Ri & \text{if } Ri \geq 0.2 \end{cases} \tag{48}$$

with an upper cap $Pr_t \leq 10$ to prevent excessively small scalar diffusivities.

Physical interpretation:

- **Strongly turbulent regime ($Ri < 0.2$):** Turbulence is vigorous enough to mix momentum and scalars equally efficiently, so $Pr_t = 1$ (i.e., $\kappa_m = \kappa_h$).
- **Weakly turbulent / transitional regime ($Ri \geq 0.2$):** Stratification suppresses vertical motion, making scalar mixing less efficient than momentum mixing. The ratio Pr_t increases linearly with Ri, reflecting the increasing dominance of buoyancy over shear.
- **Cap at $Pr_t = 10$:** This prevents κ_h from becoming unrealistically small in strongly stratified regions where Ri may be very large (e.g., $Ri > 10$ in the deep thermocline). The cap ensures a minimum level of scalar diffusion.

The critical Richardson number $Ri_c = 0.2$ is a classical result from atmospheric boundary layer theory, corresponding to the threshold for turbulence suppression by stratification.

10.10.2 Implementation

The TKE Prandtl number is computed in lines 561–588 of `ggl90_calc.F`:

```
1         DO j=jMin,jMax
2          DO i=iMin,iMax
3           RiNumber = MAX(Nsquare(i,j,k),0. _d 0)
4     &                /(verticalShear(i,j)+GGL90eps)
5           prTemp = 1. _d 0
6           IF ( RiNumber .GE. 0.2 _d 0 ) prTemp = 5. _d 0 *
              RiNumber
7           TKEPrandtlNumber(i,j,k) = MIN(10. _d 0,prTemp)
8          ENDDO
9         ENDDO
```

Listing 38: (ggl90_calc.F:L577–585) TKE Prandtl number

Key implementation details:

- Line 579: $Ri = \max(N^2, 0)/(S^2 + \varepsilon)$. The numerator is floored at zero to exclude convectively unstable regions ($N^2 < 0$), where the concept of a "Richardson number" is not well-defined. The denominator is floored at `GGL90eps` to avoid division by zero when $S^2 \approx 0$.
- Lines 581–582: Implement the piecewise formula (48).
- Line 583: Cap at $Pr_t = 10$.

Behavior in limiting cases:

- **Convective layer** ($N^2 < 0$): $Ri = 0 \Rightarrow Pr_t = 1$.
- **Neutral layer** ($N^2 \approx 0$, $S^2 > 0$): $Ri \approx 0 \Rightarrow Pr_t = 1$.
- **Strongly stratified layer** ($N^2 \gg S^2$): $Ri \gg 1 \Rightarrow Pr_t = 10$ (capped).
- **Zero shear** ($S^2 \approx 0$): $Ri \to \infty \Rightarrow Pr_t = 10$ (capped). This prevents κ_h from vanishing in quiescent regions, ensuring a background level of scalar diffusion.

10.10.3 Alternative formulation with IDEMIX

If the IDEMIX internal wave module is enabled (Section 11.1), the Prandtl number formula is modified to account for the contribution of internal wave breaking to scalar mixing:

```
#ifdef ALLOW_GGL90_IDEMIX
        IF ( useIDEMIX ) THEN
         DO j=jMin,jMax
          DO i=iMin,iMax
            RiNumber = MAX(Nsquare(i,j,k),0. _d 0)
     &              /(verticalShear(i,j)+GGL90eps)
            IDEMIX_RiNumber = MAX( KappaM(i,j)*Nsquare(i,j,k), 0.
                _d 0)/
     &              ( GGL90eps + IDEMIX_gTKE(i,j,k) )
            prTemp             = 6.6 _d 0 * MIN( RiNumber,
               IDEMIX_RiNumber )
            TKEPrandtlNumber(i,j,k) = MIN(10. _d 0,prTemp)
            TKEPrandtlNumber(i,j,k) = MAX( oneRL,TKEPrandtlNumber
               (i,j,k) )
          ENDDO
         ENDDO
        ELSE
#endif /* ALLOW_GGL90_IDEMIX */
```

Listing 39: (ggl90_calc.F:L563–574) TKE Prandtl number with IDEMIX

The IDEMIX-modified formula computes two Richardson numbers:

- `RiNumber`: Standard gradient Richardson number (as before).
- `IDEMIX_RiNumber`: An effective Richardson number based on the TKE production rate from internal wave dissipation.

The Prandtl number is then $Pr_t = 6.6 \times \min(Ri, Ri_{\text{IDEMIX}})$, capped at 10. This formulation reduces Pr_t (i.e., enhances scalar mixing) when internal waves contribute significantly to TKE production, reflecting the enhanced vertical mixing by wave breaking.

10.11 Eddy Diffusivity (κ_h)

Once κ_m and Pr_t are known, the eddy diffusivity for scalars (temperature and salinity) is simply:

$$\kappa_h = \frac{\kappa_m}{Pr_t} \tag{49}$$

This is computed in lines 593–598 of `ggl90_calc.F`:

```
1         DO j=jMin,jMax
2          DO i=iMin,iMax
3 C          diffusivity
4            KappaH = KappaM(i,j)/TKEPrandtlNumber(i,j,k)
5            KappaE(i,j,k) = GGL90alpha * KappaM(i,j)
6       &                  * maskC(i,j,k,bi,bj)*maskC(i,j,k-1,bi,bj)
```

Listing 40: (ggl90_calc.F:L593–598) Eddy diffusivity and TKE diffusivity

Note on variable naming. The local variable `KappaH` is computed but not stored; instead, the scalar diffusivities are computed later from `GGL90visctmp` (the viscosity floored at the scalar background diffusivity) in the routines `GGL90_CALC_DIFF` (Section 12).

The second line computes $\kappa_e = \alpha_e \kappa_m$, the diffusivity of TKE itself (Section 10.6), where $\alpha_e =$ `GGL90alpha` (typically 1.0).

10.11.1 Typical values

Continuing the examples from Section 10.9:

- **Surface mixed layer** ($Ri < 0.2$): $\kappa_m \sim 0.06$ m^2 s^{-1}, $Pr_t = 1 \Rightarrow \kappa_h \sim 0.06$ m^2 s^{-1}.
- **Thermocline** ($Ri \sim 1$): $\kappa_m \sim 3 \times 10^{-4}$ m^2 s^{-1}, $Pr_t = 5 \Rightarrow \kappa_h \sim 6 \times 10^{-5}$ m^2 s^{-1}.
- **Deep ocean** ($Ri \sim 5$): $\kappa_m \sim 5 \times 10^{-4}$ m^2 s^{-1}, $Pr_t = 10$ (capped) $\Rightarrow \kappa_h \sim 5 \times 10^{-5}$ m^2 s^{-1}.

The ratio κ_m/κ_h ranges from 1 (neutral/convective) to 10 (strongly stratified), consistent with atmospheric and oceanic observations.

10.12 Background Floors and Maximum Caps

To ensure numerical stability and physical realism, the GGL90 scheme imposes minimum (background) values on the eddy coefficients, preventing them from vanishing in quiescent regions where TKE is very small.

10.12.1 Background viscosity

The eddy viscosity κ_m is floored at the background vertical viscosity `viscArNr(k)`:

$$\kappa_m^{\text{total}} = \max(\kappa_m^{\text{GGL90}}, \kappa_m^{\text{background}}) \tag{50}$$

where $\kappa_m^{\text{background}}$ = `viscArNr(k)` is a depth-dependent background viscosity specified in the main model's `data` file. Typical values are $\kappa_m^{\text{background}} = 10^{-4}$ to 10^{-3} m^2 s^{-1} in the upper ocean, increasing to 10^{-3} to 10^{-2} m^2 s^{-1} in the deep ocean to represent mixing by internal waves, double diffusion, and other sub-grid-scale processes not explicitly captured by GGL90.

This is implemented in lines 461–463 of `ggl90_calc.F` (shown in Section 10.9).

10.12.2 Background diffusivity

Similarly, the eddy diffusivity for scalars is floored at `diffKrNrS(k)`:

$$\kappa_h^{\text{total}} = \max(\kappa_h^{\text{GGL90}}, \kappa_h^{\text{background}}) \tag{51}$$

where $\kappa_h^{\text{background}}$ = `diffKrNrS(k)` is typically 10^{-5} to 10^{-4} m^2 s^{-1} in the upper ocean, representing molecular diffusion and small-scale mixing processes.

Importantly, the background floor for scalars is applied to `GGL90visctmp` (line 456–458 of `ggl90_calc.F`), which is used later to compute the scalar diffusivities in `GGL90_CALC_DIFF`. This ensures that the scalar diffusivity is never less than the background, even in strongly stratified regions where Pr_t may be large.

10.12.3 Coordinate transformation

In pressure coordinates, the background viscosities and diffusivities must be scaled by `recip_coordFac*rec` to convert from z-coordinate units (m^2 s^{-1}) to pressure-coordinate units (Pa^2 s^{-1} or equivalent). This is automatically applied in lines 457 and 462 of `ggl90_calc.F`.

10.12.4 Maximum cap (optional)

MITgcm does not impose an explicit maximum cap on GGL90 eddy coefficients by default. However, users can add such a cap via the parameter `viscArMax` in the main model's `data` file, which applies globally to all viscosity schemes (not just GGL90). This is rarely necessary in practice, as the mixing length constraints (Section 9.4) and TKE dissipation (Section 10.5) naturally limit the growth of κ_m and κ_h.

10.12.5 Masking at dry interfaces

All eddy coefficients are multiplied by `maskC(i,j,k)*maskC(i,j,k-1)` to ensure they are zero at dry interfaces (e.g., adjacent to land, ice, or outside the domain). This prevents spurious fluxes across solid boundaries.

11 Additional Mixing Processes

GGL90 supports optional enhancements: the IDEMIX internal wave energy model and Langmuir circulation parameterization.

11.1 IDEMIX Extension

IDEMIX (Internal Wave Dissipation and MIXing) is an optional extension to GGL90 that parameterizes the generation, propagation, and dissipation of internal wave energy. When enabled (`ALLOW_GGL90_IDEMIX` in `GGL90_OPTIONS.h`), IDEMIX adds internal wave breaking as an additional source term in the TKE equation.

11.2 Overview of Internal Wave Model

IDEMIX represents the internal wave field as a single prognostic variable E (internal wave energy per unit mass, $\mathrm{m}^2\ \mathrm{s}^{-2}$), which evolves according to:

$$\frac{\partial E}{\partial t} = F_{\text{sources}} - \frac{E}{\tau_v} + \frac{\partial}{\partial z}\left(\kappa_E \frac{\partial E}{\partial z}\right) + \nabla_h \cdot (\kappa_{E,h} \nabla_h E) \tag{52}$$

where:

- F_{sources}: Energy input from tidal forcing and near-inertial wind forcing
- τ_v: Vertical dissipation time scale (typically 2–10 days)
- κ_E, $\kappa_{E,h}$: Vertical and horizontal diffusivities of internal wave energy

The dissipated internal wave energy is converted to TKE with an efficiency factor γ_{mix} (typically 0.1–0.2).

11.3 Coupling to GGL90

IDEMIX couples to GGL90 through two mechanisms:

1. TKE source term: The internal wave dissipation rate $\varepsilon_{\text{IW}} = E/\tau_v$ is added to the TKE equation (Section 10.1):

$$\frac{\partial e}{\partial t} = \ldots + \gamma_{\text{mix}} \frac{E}{\tau_v} \tag{53}$$

This enhances mixing in the ocean interior where internal wave breaking is significant (e.g., over rough topography, at the equator).

2. Modified Prandtl number: The turbulent Prandtl number formula is modified to account for internal wave-driven mixing (Section 10.10), reducing Pr_t and enhancing scalar diffusivity when E is large.

When to use IDEMIX: IDEMIX is beneficial for global ocean simulations where interior mixing from internal tides and near-inertial waves is important for water mass properties and overturning circulation. It requires additional forcing files (`IDEMIX_tidal_file`, `IDEMIX_wind_file`) and runtime parameters in the `GGL90_PARM02` namelist. For regional or process studies focused on surface boundary layer mixing, IDEMIX can be omitted.

12 Eddy Coefficients and Model Interface

After GGL90 computes vertical mixing coefficients, these must be transferred to the main model.

The GGL90 package communicates with the main MITgcm model through a set of interface routines that transfer eddy coefficients from the GGL90 arrays to the main model's mixing arrays. These routines are called during the vertical mixing phase of the time-stepping sequence.

Three key interface routines handle this communication:

1. `GGL90_CALC_VISC`: Transfers eddy viscosity to momentum equations (Section 12.1).
2. `GGL90_CALC_DIFF`: Transfers eddy diffusivity to tracer equations (Section 12.2).
3. `GGL90_EXCHANGES`: Updates halo regions for TKE when horizontal diffusion is active (Section 12.3).

These routines are designed to be minimal and efficient, adding only the GGL90 contribution to the existing background mixing coefficients already initialized by the main model.

12.1 `GGL90_CALC_VISC`: Viscosity Transfer

12.1.1 Purpose

The subroutine `GGL90_CALC_VISC` transfers the GGL90 eddy viscosity to the main model's momentum solver. It is called once per vertical level during the momentum equation time-stepping.

12.1.2 Call signature

```
      SUBROUTINE GGL90_CALC_VISC(
     I                bi, bj, iMin, iMax, jMin, jMax, k,
     U                KappaRU, KappaRV,
     I                myThid )
```

Listing 41: (ggl90_calc_visc.F:L7–10) Subroutine interface

Arguments:

- `bi, bj`: Current tile indices (for domain decomposition).
- `iMin, iMax, jMin, jMax`: Horizontal index range for the computation.
- `k`: Vertical level index (1 to Nr).
- `KappaRU, KappaRV`: Input/output arrays for vertical viscosity at U-points and V-points ($m^2\ s^{-1}$).
- `myThid`: Thread ID number (for parallel execution).

12.1.3 Implementation

The routine is remarkably simple, consisting of just two loops that add the GGL90 viscosity increment to the existing background viscosity:

```
1       DO j=jMin,jMax
2        DO i=iMin,iMax
3         KappaRU(i,j,k) = KappaRU(i,j,k)
4      &                 + ( GGL90viscArU(i,j,k,bi,bj) - viscArNr
         (k) )
5        ENDDO
6       ENDDO
7
8       DO j=jMin,jMax
9        DO i=iMin,iMax
10        KappaRV(i,j,k) = KappaRV(i,j,k)
11     &                 + ( GGL90viscArV(i,j,k,bi,bj) - viscArNr
         (k) )
12       ENDDO
13      ENDDO
```

Listing 42: (ggl90_calc_visc.F:L47–59) Add GGL90 viscosity to momentum arrays

Key details:

- `KappaRU`, `KappaRV` on entry contain the background viscosity `viscArNr(k)` (set by the main model).
- The increment `GGL90viscArU - viscArNr(k)` represents the *additional* viscosity from GGL90, beyond the background.
- This ensures that the total viscosity is $\kappa_{\text{total}} = \kappa_{\text{background}} + (\kappa_{\text{GGL90}} - \kappa_{\text{background}}) = \kappa_{\text{GGL90}}$ when GGL90 is active, but reverts to $\kappa_{\text{background}}$ in cells where GGL90 produces less mixing than the background.
- The subtraction of `viscArNr(k)` is necessary because `GGL90viscArU` and `GGL90viscArV` already include the background floor (see Section 10.12).

12.1.4 Location in time-stepping sequence

`GGL90_CALC_VISC` is called from the main momentum routine `MOM_VECINV` (or `MOM_FLUXFORM`, depending on the momentum advection scheme) at each vertical level, immediately before the vertical viscosity term is applied. This allows GGL90 to override the background viscosity on a level-by-level basis.

12.2 `GGL90_CALC_DIFF`: Diffusivity Transfer

12.2.1 Purpose

The subroutine `GGL90_CALC_DIFF` transfers the GGL90 eddy diffusivity to the main model's tracer solver. Unlike `GGL90_CALC_VISC`, which is called separately for U and V velocities, this routine handles diffusivity for all scalar tracers (temperature, salinity, and any passive tracers) with a single call.

12.2.2 Call signature

```
1       SUBROUTINE GGL90_CALC_DIFF(
2      I           bi, bj, iMin, iMax, jMin, jMax, kArg, kSize,
3      U           KappaRx,
4      I           myThid )
```

Listing 43: (ggl90_calc_diff.F:L7–10) Subroutine interface

Arguments:

- `bi, bj`: Current tile indices.
- `iMin, iMax, jMin, jMax`: Horizontal index range.
- `kArg`: Vertical level index. If `kArg = 0`, process all levels; if `kArg > 0`, process only level `kArg`.
- `kSize`: Third dimension of the `KappaRx` array (usually `Nr`, but may differ in some configurations).
- `KappaRx`: Input/output array for vertical diffusivity ($m^2\ s^{-1}$).
- `myThid`: Thread ID number.

12.2.3 Implementation

The routine supports two calling modes: process all levels, or process a single level.

Mode 1: Process all levels (`kArg = 0`)

```
      IF ( kArg .EQ. 0 ) THEN
C--   do all levels :
        DO k=1,MIN(Nr,kSize)
         DO j=jMin,jMax
          DO i=iMin,iMax
            KappaRx(i,j,k) = KappaRx(i,j,k)
     &                     +( GGL90diffKr(i,j,k,bi,bj)
     &                        - diffKrNrS(k) )
          ENDDO
         ENDDO
        ENDDO
```

Listing 44: (ggl90_calc_diff.F:L48–58) Add GGL90 diffusivity: all levels

Mode 2: Process single level (`kArg > 0`)

```
      ELSE
C--   do level k=kArg only :
        k = MIN(kArg,kSize)
        DO j=jMin,jMax
         DO i=iMin,iMax
            KappaRx(i,j,k) = KappaRx(i,j,k)
     &                     +( GGL90diffKr(i,j,kArg,bi,bj)
     &                        - diffKrNrS(kArg) )
          ENDDO
        ENDDO
      ENDIF
```

Listing 45: (ggl90_calc_diff.F:L59–69) Add GGL90 diffusivity: single level

Key details:

- Similar to `GGL90_CALC_VISC`, the routine adds the GGL90 diffusivity increment `GGL90diffKr - diffK` to the existing background diffusivity.
- `diffKrNrS(k)` is the background scalar diffusivity (typically 10^{-5} to 10^{-4} m^2 s^{-1} in the upper ocean).
- The two-mode design allows the main model to call `GGL90_CALC_DIFF` either once per time step (all levels at once) or once per level (level-by-level), depending on the tracer advection scheme.

12.3 `GGL90_EXCHANGES`: Halo Updates

12.3.1 Purpose

The subroutine `GGL90_EXCHANGES` updates the halo (overlap) regions of the TKE array `GGL90TKE` to ensure consistency across tile boundaries in domain-decomposed simulations. This is necessary when horizontal diffusion of TKE is active (`GGL90diffTKEh > 0`) or when the optional IDEMIX package is used with horizontal internal wave energy diffusion.

12.3.2 Implementation

```
#ifdef ALLOW_GGL90_IDEMIX
      IF ( useIDEMIX .AND. IDEMIX_tau_h .GT. zeroRL ) THEN
       _EXCH_XYZ_RL( GGL90TKE, myThid )
       _EXCH_XYZ_RL( IDEMIX_E, myThid )
      ELSEIF ( GGL90diffTKEh .GT. zeroRL ) THEN
#else /* ALLOW_GGL90_IDEMIX */
      IF ( GGL90diffTKEh .GT. zeroRL ) THEN
#endif /* ALLOW_GGL90_IDEMIX */
       _EXCH_XYZ_RL( GGL90TKE, myThid )
      ENDIF
```

Listing 46: (ggl90_exchanges.F:L31–40) Conditional halo exchanges

Key details:

- `_EXCH_XYZ_RL` is a MITgcm macro that expands to the appropriate halo exchange routine for 3D real*8 arrays.
- Halo exchanges are *only* performed when horizontal TKE diffusion or IDEMIX horizontal diffusion is enabled.
- If neither feature is active, the routine is a no-op (does nothing).

13 Initialization and Restart

GGL90 initialization proceeds in three stages: reading runtime parameters (Section 13.1), initializing time-invariant fields (Section 13.2), and initializing time-varying fields including restart capability (Section 13.3).

13.1 `GGL90_READPARMS`: Parameter Input

The routine `GGL90_READPARMS` reads runtime parameters from the file `data.ggl90` in the run directory. This file contains three Fortran namelists:

- `GGL90_PARM01`: Core GGL90 parameters (required)

- `GGL90_PARM02`: IDEMIX parameters (optional, only if `ALLOW_GGL90_IDEMIX` is defined)
- `GGL90_PARM03`: Langmuir circulation parameters (optional, only if `ALLOW_GGL90_LANGMUIR` is defined)

Default values (from `ggl90_readparms.F:L102--127`):

- `GGL90ck = 0.1`: Eddy viscosity constant c_μ
- `GGL90ceps = 0.7`: Dissipation constant c_ε
- `GGL90alpha = 1.0`: TKE diffusivity constant α_e
- `GGL90m2 = 3.75`: Surface TKE constant B_0 (originally from Blanke & Delecluse 1993)
- `GGL90TKEmin = 1e-11`: Minimum TKE ($\mathrm{m}^2\ \mathrm{s}^{-2}$)
- `GGL90TKEsurfMin = 1e-4`: Minimum surface TKE ($\mathrm{m}^2\ \mathrm{s}^{-2}$)
- `GGL90mixingLengthMin = 1e-8`: Minimum mixing length (m)
- `mxlMaxFlag = 0`: Mixing length method (0, 1, or 2)
- `mxlSurfFlag = .FALSE.`: Force surface mixing length to grid spacing
- `GGL90_dirichlet = .TRUE.`: Use Dirichlet bottom BC for TKE
- `calcMeanVertShear = .FALSE.`: Method for computing vertical shear

13.2 GGL90_INIT_FIXED: Time-Invariant Initialization

The routine `GGL90_INIT_FIXED` initializes fields that do not change during the simulation. This is a minimal routine that primarily registers diagnostics:

```
1 #ifdef ALLOW_DIAGNOSTICS
2       IF ( useDiagnostics ) THEN
3         CALL GGL90_DIAGNOSTICS_INIT( myThid )
4       ENDIF
5 #endif
```

Listing 47: (ggl90_init_fixed.F:L39–43) Diagnostics registration

If horizontal smoothing is enabled (`ALLOW_GGL90_SMOOTH`), the routine also initializes a mask array for handling cubed-sphere corner points (lines 45–63).

13.3 GGL90_INIT_VARIA: Time-Varying Field Initialization

The routine `GGL90_INIT_VARIA` initializes all GGL90 prognostic and diagnostic arrays at the start of a simulation. The key steps are:

1. Zero out eddy coefficient arrays:

```
1           DO k=1,Nr
2            DO j=1-OLy,sNy+OLy
3             DO i=1-OLx,sNx+OLx
4               GGL90viscArU(i,j,k,bi,bj) = 0. _d 0
5               GGL90viscArV(i,j,k,bi,bj) = 0. _d 0
6               GGL90diffKr(i,j,k,bi,bj)  = 0. _d 0
7               IF ( useIDEMIX) THEN
8                GGL90TKE(i,j,k,bi,bj)=GGL90eps*maskC(i,j,k,bi,bj)
9               ELSE
10               GGL90TKE(i,j,k,bi,bj)=GGL90TKEmin*maskC(i,j,k,bi,bj
                     )
11              ENDIF
12             ENDDO
13            ENDDO
14           ENDDO
```

Listing 48: (ggl90_init_varia.F:L42–56) Initialize arrays to zero

Note that TKE is initialized to `GGL90TKEmin` (or `GGL90eps` if IDEMIX is active), not zero, to avoid division by zero in the first time step.

2. Read initial TKE from file (optional): If the parameter `GGL90TKEFile` is specified and this is not a restart run:

```
1           IF ( GGL90TKEFile .NE. ' ' ) THEN
2            CALL READ_FLD_XYZ_RL( GGL90TKEFile, ' ', GGL90TKE, 0,
                 myThid )
3            _EXCH_XYZ_RL(GGL90TKE,myThid)
4            DO bj=myByLo(myThid),myByHi(myThid)
5             DO bi=myBxLo(myThid),myBxHi(myThid)
6              DO k=1,Nr
7               DO j=1-OLy,sNy+OLy
8                DO i=1-OLx,sNx+OLx
9                 GGL90TKE(i,j,k,bi,bj) = MAX(GGL90TKE(i,j,k,bi,bj),
10      &                GGL90TKEmin)*maskC(i,j,k,bi,bj)
11               ENDDO
12              ENDDO
13             ENDDO
14            ENDDO
15           ENDDO
16          ENDIF
```

Listing 49: (ggl90_init_varia.F:L135–150) Read initial TKE from file

The file should contain a 3D field of TKE values ($m^2\ s^{-2}$) in MITgcm's standard binary format.

3. Read restart pickup file: If this is a restart run (`nIter0 != 0` or `pickupSuff != ' '`), load the prognostic TKE field from the pickup file:

```
1       IF ( nIter0.NE.0 .OR. pickupSuff.NE.' ' ) THEN
2        CALL GGL90_READ_PICKUP( nIter0, myThid )
3       ELSE
```

Listing 50: (ggl90_init_varia.F:L131–133) Load from restart

13.4 Restart Files: GGL90_READ/WRITE_PICKUP

GGL90 uses MITgcm's standard pickup file mechanism to save and restore the prognostic TKE field during restart.

Pickup file contents:

- `GGL90TKE`: 3D TKE field ($m^2\ s^{-2}$)
- `IDEMIX_E`: 3D internal wave energy field ($m^2\ s^{-2}$), only if `ALLOW_GGL90_IDEMIX` is active

File naming convention:

- Standard restart: `pickup_ggl90.ITERATION.data` and `.meta`
- Checkpoint: `pickup_ggl90.ckptA.data` and `.meta` (or `.ckptB`)

Usage: No user action is required. MITgcm automatically calls `GGL90_WRITE_PICKUP` at the end of each run and `GGL90_READ_PICKUP` at the start of a restart run.

14 Compile-Time Options

GGL90 behavior is controlled by a combination of runtime parameters (in `data.ggl90`) and compile-time CPP flags (in `GGL90_OPTIONS.h`). This section documents the available compile-time options.

14.1 Core Options

`ALLOW_GGL90` Master flag to enable the GGL90 package. Must be defined in `packages.conf` to compile GGL90 routines. If not defined, all GGL90 code is excluded from compilation.

`GGL90_REGULARIZE_MIXINGLENGTH` If defined, use a regularized formula for the minimum mixing length (Section 9.5):

$$\ell_{\text{final}} = \sqrt{\ell^2 + \ell_{\text{min}}^2}$$

instead of the standard $\max(\ell, \ell_{\text{min}})$. This provides smoother derivatives for adjoint calculations (see Appendix **??**). Default: *not defined*.

`GGL90_MISSING_HFAC_BUG` Legacy flag related to partial cell thickness handling in IDEMIX. Retains a historical bug for backward compatibility with older configurations (see Appendix **??**). Default: *not defined*. New users should *not* define this.

14.2 Optional Feature Flags

`ALLOW_GGL90_HORIZDIFF` Enable horizontal diffusion of TKE. When defined, the runtime parameter `GGL90diffTKEh` controls the horizontal diffusivity ($m^2\ s^{-1}$). Default: *not defined*. This feature is rarely used in practice (see Section 10.7).

`ALLOW_GGL90_SMOOTH` Enable horizontal smoothing of eddy viscosity and diffusivity before transferring to the main model. Uses a 9-point stencil following the OPA/NEMO approach. Default: *not defined*. Useful for coarse-resolution models to reduce grid-scale noise.

`ALLOW_GGL90_IDEMIX` Enable the IDEMIX internal wave energy parameterization (Section 11.1). Adds internal wave breaking as a TKE source. Default: *not defined*. Requires additional parameters in `GGL90_PARM02` namelist.

`ALLOW_GGL90_LANGMUIR` Enable Langmuir circulation parameterization. Enhances surface layer mixing under wind-wave forcing. Default: *not defined*. Requires additional parameters in `GGL90_PARM03` namelist.

15 Output and Diagnostics

GGL90 provides a comprehensive set of diagnostic fields for monitoring mixing processes. All diagnostics are registered via MITgcm's standard diagnostics package.

15.1 Available Diagnostic Fields

Table 7 lists the available GGL90 diagnostic fields.

Table 7: GGL90 diagnostic fields

Name	Dims	Description	Units
`GGL90TKE`	3D	Turbulent kinetic energy	$m^2\ s^{-2}$
`GGL90ArU`	3D	Eddy viscosity at U-points	$m^2\ s^{-1}$
`GGL90ArV`	3D	Eddy viscosity at V-points	$m^2\ s^{-1}$
`GGL90Kr`	3D	Eddy diffusivity (scalars)	$m^2\ s^{-1}$
`GGL90mixL`	3D	Mixing length	m
`GGL90Prandtl`	3D	Turbulent Prandtl number	–
`GGL90flx`	2D	Surface TKE flux	$m^3\ s^{-3}$
`GGL90tau`	2D	Surface friction velocity	$m\ s^{-1}$

Usage example: To output hourly snapshots of TKE and eddy diffusivity, add to `data.diagnostics`:

```
fields(1:2,1) = 'GGL90TKE','GGL90Kr ',
filename(1) = 'ggl90_snap',
frequency(1) = 3600.0,
```

15.2 Diagnostics Registration

Diagnostics are registered in `GGL90_DIAGNOSTICS_INIT`, called from `GGL90_INIT_FIXED`. The registration is automatic and requires no user action. To enable diagnostics output, set `useDiagnostics = .TRUE.` in the main `data` file and configure the desired fields in `data.diagnostics`.

16 Package Validation

The routine `GGL90_CHECK` performs compile-time and runtime validation of GGL90 parameters. It is called during model initialization to catch common configuration errors before the simulation begins.

Checks performed:

- All required runtime parameters have been set (not `UNSET_RL`)
- Physical constants are positive (e.g., `GGL90ck > 0`, `GGL90TKEmin > 0`)
- Mixing length method `mxlMaxFlag` is valid (0, 1, or 2)
- Background mixing coefficients are physically reasonable
- If IDEMIX is enabled, all IDEMIX parameters are set

If any check fails, the routine prints an error message and stops the model with `STOP 'ABNORMAL END'`. This prevents silent failures due to misconfiguration.

User action: If `GGL90_CHECK` reports an error, review the parameters in `data.ggl90` and correct the issue. Common problems include typos in parameter names (Fortran namelists are case-insensitive but require exact spelling) and missing parameters when optional features are enabled.

17 Numerical Considerations & Frequently Encountered Issues

This section provides practical guidance for diagnosing and resolving common GGL90 problems.

17.1 Parameter Tuning Guidance

Example starting configuration: For a well-tested baseline configuration suitable for global ocean applications, see Appendix **??**. This provides a complete `data.ggl90` file with physically motivated parameter choices.

Tuning for specific applications:

- **Too much mixing:** Reduce `GGL90ck` (e.g., 0.08) or increase `GGL90ceps` (e.g., 0.9).
- **Too little mixing:** Increase `GGL90ck` (e.g., 0.12) or decrease `GGL90ceps` (e.g., 0.5).
- **Unrealistic surface mixed layer depth:** Adjust `GGL90m2` (surface TKE constant). Lower values give shallower mixed layers.
- **Numerical instability:** Ensure `GGL90TKEmin > 1e-8` and `GGL90mixingLengthMin > 1e-10`.

17.2 Common Mistakes and Solutions

1. Forgetting to enable GGL90 in data file: **Symptom:** Model runs but mixing is weak, diagnostics show zero TKE.
Solution: Set `useGGL90 = .TRUE.` in the main `data` file (not `data.ggl90`).

2. Incompatible coordinate system: **Symptom:** Mixing length values are unrealistically small or large.
Solution: GGL90 handles z- and p-coordinates automatically via `coordFac`. No user action needed, but verify that `drF` and `drC` are correctly set in `data.grid`.

3. Missing pickup file on restart: **Symptom:** Model crashes with "file not found: pickup_ggl90.XXX.data".
Solution: Ensure the pickup file from the previous run is in the run directory. If starting from a checkpoint that predates GGL90 activation, set `nIter0 = 0` to cold-start GGL90.

4. Conflicting mixing schemes: **Symptom:** Excessive or oscillatory mixing, conservation errors.
Solution: Disable other vertical mixing schemes (KPP, PP81, MY82) when using GGL90. Only one turbulence closure should be active at a time.

17.3 Performance Considerations

Computational cost: GGL90 adds approximately 10–20% to the total runtime of a typical ocean simulation, dominated by:

- TKE implicit solve (tridiagonal system, one solve per horizontal point per time step)
- Mixing length computation (especially Method 2 with two vertical sweeps)
- Eddy coefficient interpolation to velocity points

Optimization strategies:

- Use Method 1 instead of Method 2 for mixing length if the extra accuracy is not needed (saves 30% of GGL90 cost).
- Disable horizontal TKE diffusion (`GGL90diffTKEh = 0`, the default) to avoid halo exchanges.
- For adjoint calculations, see Appendix **??** for optimization-specific settings and tuning recommendations.

17.4 Minimum Alpha for Oscillation-Free Solutions

17.4.1 Motivation: Why Alpha Matters for Numerical Accuracy

While GGL90 uses fully implicit time-stepping for TKE diffusion (ensuring unconditional numerical stability), the choice of α (turbulent Prandtl number) has a critical impact on the **accuracy** and **smoothness** of the solution. Small values of α can produce cell-to-cell oscillations in TKE and mixing coefficients, even though the simulation remains stable.

The physical mechanism is:

- Small $\alpha \Rightarrow$ small $\kappa_h = \kappa_m/\alpha \Rightarrow$ weak TKE diffusion
- Weak TKE diffusion $\Rightarrow$ sharp, localized TKE gradients
- Sharp gradients on finite-difference grid $\Rightarrow$ under-resolved features and oscillations

This is a **resolution issue**, not a stability issue. The implicit treatment of diffusion eliminates CFL-like constraints, but cannot create information at scales finer than the grid spacing.

17.4.2 Oscillation Diagnosis Results

Systematic testing of the arctic convection scenario (strong cooling, ice formation, deep convection) across $\alpha = 1, 3, 5, 7.5, 10, 15, 20, 30, 50$ reveals:

Key finding: Three independent metrics agree on a threshold of $\alpha \geq 5$. TKE roughness drops by a factor of $\sim$27 from $\alpha = 1$ to $\alpha = 5$ (falling below 10% of its maximum); the oscillation count falls from 11 to 3; and the fraction of layers whose gradient length scale $L = |\mathrm{TKE}|/|\partial \mathrm{TKE}/\partial z|$ is under-resolved ($L < 2\Delta z$) drops from 0.53 to 0.06. The last metric directly confirms the mechanism: at $\alpha = 1$ roughly half the column is under-resolved, producing the cell-to-cell oscillations, whereas by $\alpha = 5$ almost the entire column is well resolved.

Note on metric choice: the resolution ratio is reported here as the *median* $L/\Delta z$ together with the *fraction* of under-resolved layers. A single minimum $L/\Delta z$ is not informative because a TKE profile spans many orders of magnitude, so its steepest single transition drives the minimum toward zero at every α; the median and the under-resolved fraction are the robust measures.

Table 8: TKE oscillation metrics vs. α for arctic convection scenario

α	**TKE Roughness** $\sum \lvert\partial^2 \mathrm{TKE}/\partial z^2\rvert$	**Normalized Roughness**	**Oscillation Count**	**Under-resolved fraction** ($L < 2\Delta z$)	**Status**
1.0	1.22×10^{-2}	1.961	11	0.53	OSCILLATORY
3.0	4.53×10^{-3}	0.959	9	0.27	OSCILLATORY
5.0	4.61×10^{-4}	0.128	3	0.06	THRESHOLD
7.5	4.12×10^{-4}	0.112	3	0.06	SMOOTH
10.0	3.82×10^{-4}	0.103	3	0.12	SMOOTH
15.0	3.40×10^{-4}	0.090	3	0.12	SMOOTH
20.0	3.09×10^{-4}	0.081	1	0.12	SMOOTH
30.0	2.61×10^{-4}	0.068	1	0.12	SMOOTH
50.0	2.15×10^{-4}	0.055	1	0.12	SMOOTH

17.4.3 Why ECCOv4 Uses $\alpha = 30$

ECCOv4's choice of $\alpha = 30$ is well-justified:

- **Safety factor:** $30/5 = 6\times$ above the oscillation threshold
- **Physical realism:** Observational studies suggest $\alpha \sim 20$–50 in stratified ocean interior
- **Adjoint stability:** Smoother mixing fields reduce gradient noise in adjoint model
- **Thermocline preservation:** Reduced κ_h prevents excessive erosion of seasonal thermocline

17.4.4 Practical Recommendations

For new GGL90 applications:

1. **Minimum:** Use $\alpha \geq 5$ to avoid numerical oscillations
2. **Standard:** Use $\alpha = 10$–20 for most regional/process studies
3. **Global/adjoint:** Use $\alpha = 30$ (ECCOv4 value) for state estimation
4. **High stratification:** Consider $\alpha = 50$ for strongly stratified regimes

Grid dependence: The minimum α is set by how well the grid resolves the TKE gradient length scale L, not by a stability (grid-Reynolds) limit — the implicit solve is unconditionally stable. What matters is the ratio $L/\Delta z$: since larger α increases κ_h and spreads TKE (increasing L), the threshold depends on the grid and forcing, so re-testing is recommended for a new configuration.

Testing protocol: When adapting GGL90 to a new configuration, plot TKE and κ_m vertical profiles. If you observe cell-to-cell oscillations, increase α until profiles are smooth.

18 Conclusions

This document provides a comprehensive technical reference for the GGL90 turbulence closure scheme as implemented in the MIT General Circulation Model (MITgcm). We have documented:

1. **Physical foundation:** The TKE-based turbulence closure theory of Gaspar et al. (1990), including the governing equations, mixing length formulation, and eddy coefficient parameterization (Sections **??**–10.8).

2. **Numerical implementation:** Complete line-by-line correspondence with MITgcm source code (`pkg/ggl90/*.F`), covering initialization, time integration, boundary conditions, and model interfaces (Sections **??**–15).

3. **Input/output contracts:** Comprehensive tables and data flow diagram documenting all exchanges between GGL90 and the main model (Appendix A).

4. **Practical guidance:** Parameter tuning recommendations, common issues, and performance considerations (Section 17).

Key design features of GGL90:

- **Prognostic TKE:** Solves a transport equation for turbulent kinetic energy, allowing the turbulence field to respond dynamically to changing forcing and stratification.

- **Mixing length from stability:** Computes the vertical scale of turbulent eddies from local TKE and buoyancy frequency, with geometric constraints preventing unphysical penetration across stable layers (Method 2, Blanke & Delecluse 1993).

- **Richardson number-dependent Prandtl number:** Reduces scalar diffusivity relative to momentum diffusivity in stratified regions, consistent with atmospheric and oceanic observations.

- **Clean model interface:** GGL90 receives velocity, density gradient, and wind stress as inputs, and returns eddy coefficients as outputs, with no direct access to tracers or other model internals. This modular design facilitates testing, comparison with other schemes, and portability to other ocean models.

Recommendations for users:

- Start with a well-validated configuration (e.g., Appendix **??** for global applications) as a baseline.

- Monitor TKE and eddy coefficient diagnostics (`GGL90TKE`, `GGL90Kr`) to verify that mixing behavior is physically reasonable.

- Compare against observations (e.g., Argo mixed layer depth, mooring time series) when tuning parameters for regional applications.

- Consult the original references (Gaspar et al. 1990; Blanke & Delecluse 1993) for the underlying physics and theory.
- For adjoint-based applications, see Appendix **??** for gradient-specific considerations.

Future development: Potential enhancements to GGL90 include:

- Bottom boundary layer parameterization with explicit bottom friction velocity
- Langmuir turbulence with Stokes drift from a wave model (already available via `ALLOW_GGL90_LANGMU`
- Anisotropic mixing length for submesoscale-resolving simulations
- Integration with sea ice models for under-ice turbulence

The GGL90 package remains under active development within the MITgcm community, with ongoing refinements for global ocean state estimation and high-resolution process studies.

19 Glossary of Symbols

This glossary defines the most frequently used mathematical symbols and variable names in the GGL90 documentation and MITgcm source code.

Physical Variables

Symbol	Code Variable	Description	Units
e	`GGL90TKE`	Turbulent kinetic energy per unit mass	$\mathrm{m}^2\ \mathrm{s}^{-2}$
ℓ	`GGL90mixingLength`	Mixing length (vertical scale of turbulent eddies)	m
κ_m	`GGL90viscArU`, `GGL90viscArV`	Eddy viscosity for momentum	$\mathrm{m}^2\ \mathrm{s}^{-1}$
κ_h	`GGL90diffKr`	Eddy diffusivity for scalars (T, S)	$\mathrm{m}^2\ \mathrm{s}^{-1}$
κ_e	`KappaE`	Eddy diffusivity for TKE itself	$\mathrm{m}^2\ \mathrm{s}^{-1}$
N^2	`Nsquare`	Squared buoyancy frequency (stratification), $N^2 = -(g/\rho_0)\partial\rho/\partial z$	s^{-2}
S^2	`verticalShear`	Squared vertical shear of horizontal velocity, $S^2 = (\partial u/\partial z)^2 + (\partial v/\partial z)^2$	s^{-2}

Symbol	Code Variable	Description	Units
Ri	`RiNumber`	Richardson number, $Ri = N^2/S^2$	–
Pr_t	`TKEPrandtlNumber`	Turbulent Prandtl number, $Pr_t = \kappa_m/\kappa_h$	–
u_*	`uStarSquare` (actually u_* after sqrt)	Friction velocity at surface or bottom	m s^{-1}
ρ	(not stored in GGL90)	In-situ density	kg m^{-3}
$\partial\rho/\partial z$	`sigmaR`	Vertical density gradient	kg m^{-4}

Dimensionless Constants

Symbol	Code Parameter	Description	Default
c_μ	`GGL90ck`	Eddy viscosity constant	0.1
c_ε	`GGL90ceps`	TKE dissipation constant	0.7
α_e	`GGL90alpha`	TKE diffusivity constant	1.0
B_0	`GGL90m2`	Surface TKE boundary condition constant	3.75
$e_{\min}$	`GGL90TKEmin`	Minimum TKE (floor value)	10^{-11} m^2 s^{-2}
$\ell_{\min}$	`GGL90mixingLengthMin`	Minimum mixing length (floor value)	10^{-8} m
$\sqrt{2}$	`SQRTTWO`	Constant in Gaspar mixing length formula	1.41421356

Grid and Coordinate Variables

Symbol	Code Variable	Description	Units
k	`k`	Vertical level index (1 to Nr)	–
N_r	`Nr`	Number of vertical levels	–
Δz_k	`drF(k)`	Thickness of cell k at interface	m
–	`drC(k)`	Distance between cell centers k and $k+1$	m
z	`rF(k)`	Vertical position of interface k (negative down in ocean)	m

Symbol	Code Variable	Description	Units
–	`coordFac`	Coordinate scaling factor (1 in z-coords, $g\rho_0$ in p-coords)	–
–	`maskC`, `maskU`, `maskV`	Land/ocean masks (0 = land, 1 = ocean)	–

Acronyms and Abbreviations

- **GGL90**: Gaspar-Grégoris-Lefevre 1990 turbulence closure scheme
- **TKE**: Turbulent Kinetic Energy
- **KPP**: K-Profile Parameterization (alternative mixing scheme)
- **IDEMIX**: Internal wave Dissipation and MIXing parameterization
- **MITgcm**: Massachusetts Institute of Technology General Circulation Model
- **ECCOv4**: Estimating the Circulation and Climate of the Ocean, version 4
- **CPP**: C PreProcessor (compile-time flags)
- **EOS**: Equation Of State
- **BC**: Boundary Condition
- **RHS**: Right Hand Side (of equation)
- **LHS**: Left Hand Side (of equation)

A Complete Input/Output Mapping

This appendix provides a comprehensive reference for all data exchanges between the GGL90 package and the main MITgcm model. Understanding these interfaces is essential for:

- Debugging GGL90 behavior in coupled simulations
- Implementing GGL90 ports to other ocean models
- Modifying GGL90 to use alternative input fields (e.g., from observations)
- Diagnosing conservation properties and boundary condition handling

The GGL90 package operates as a "mixing engine" that receives velocity, density, and surface forcing fields from the main model, computes eddy coefficients and updates TKE, and returns the eddy coefficients to the main model for use in momentum and tracer equations. Figure 2 summarizes this bidirectional exchange.

A.1 External Inputs from Main Model

Table 12 lists all fields that GGL90 reads from the main model. These are *inputs* from GGL90's perspective, but are computed elsewhere in MITgcm.

Key clarifications:

- **Density gradient vs. T/S:** GGL90 does *not* compute the equation of state internally. It receives the pre-computed density gradient `sigmaR` from the main model's `FIND_RHO` routine. This ensures consistency with the model's pressure gradient calculation.
- **Surface forcing sign convention:** `surfaceForcingU/V` are kinematic stresses (stress divided by ρ_0), with units of $m^2\ s^{-2}$. Positive values indicate stress in the positive U/V direction (eastward/northward in geographic coordinates).
- **Background mixing:** The arrays `viscArNr` and `diffKrNrS` are depth-dependent (varying with level index k) to allow specification of enhanced mixing in specific depth ranges (e.g., deep ocean background mixing from internal wave breaking).
- **Coordinate-independent:** GGL90 works in both z-coordinates and pressure coordinates. The `coordFac` scaling factor (computed internally from `usingPCoords`) handles the conversion.

A.2 Package Outputs to Main Model

Table 13 lists all fields that GGL90 writes for use by the main model. These are *outputs* from GGL90's perspective.

Key clarifications:

- **Single diffusivity for all tracers:** GGL90 computes one diffusivity field `GGL90diffKr` that is applied to temperature, salinity, and all passive tracers. This assumes the turbulent Prandtl number is the same for all scalars.
- **Already includes background:** The output arrays `GGL90viscArU`, `GGL90viscArV`, and `GGL90diffKr` already include the background mixing coefficients (`viscArNr`, `diffKrNrS`). The interface routines `GGL90_CALC_VISC` and `GGL90_CALC_DIFF` subtract the background to avoid double-counting (see Section 12).
- **Update timing:** GGL90 is called once per time step, after the tracer and momentum equations have been advected but before the implicit vertical mixing solve. The updated eddy coefficients are used immediately in the same time step.

A.3 Internal Prognostic State

Table 14 lists the internal prognostic variables that GGL90 carries between time steps. These are stored in the GGL90 common block and written to/read from pickup files during restart.

Key clarifications:

- **TKE is the only standard prognostic variable:** Unlike some other turbulence closures (e.g., k-ε or k-ω), GGL90 solves for TKE only. The mixing length ℓ is diagnosed algebraically from TKE and stratification (Section 9.4), not evolved prognostically.
- **Eddy coefficients are diagnostic:** `GGL90viscArU`, `GGL90viscArV`, and `GGL90diffKr` are recomputed from TKE at every time step. They are *not* prognostic variables and are *not* written to pickup files.
- **Restart behavior:** At the start of a simulation (`nIter0 = 0`), TKE is initialized to `GGL90TKEmin` or read from `GGL90TKEFile` if specified. During a restart (`nIter0 > 0`), TKE is read from the pickup file, preserving the exact state from the previous run.

A.4 Data Flow Diagram

Figure 2 illustrates the complete data flow between the main MITgcm model and the GGL90 package within a single time step.

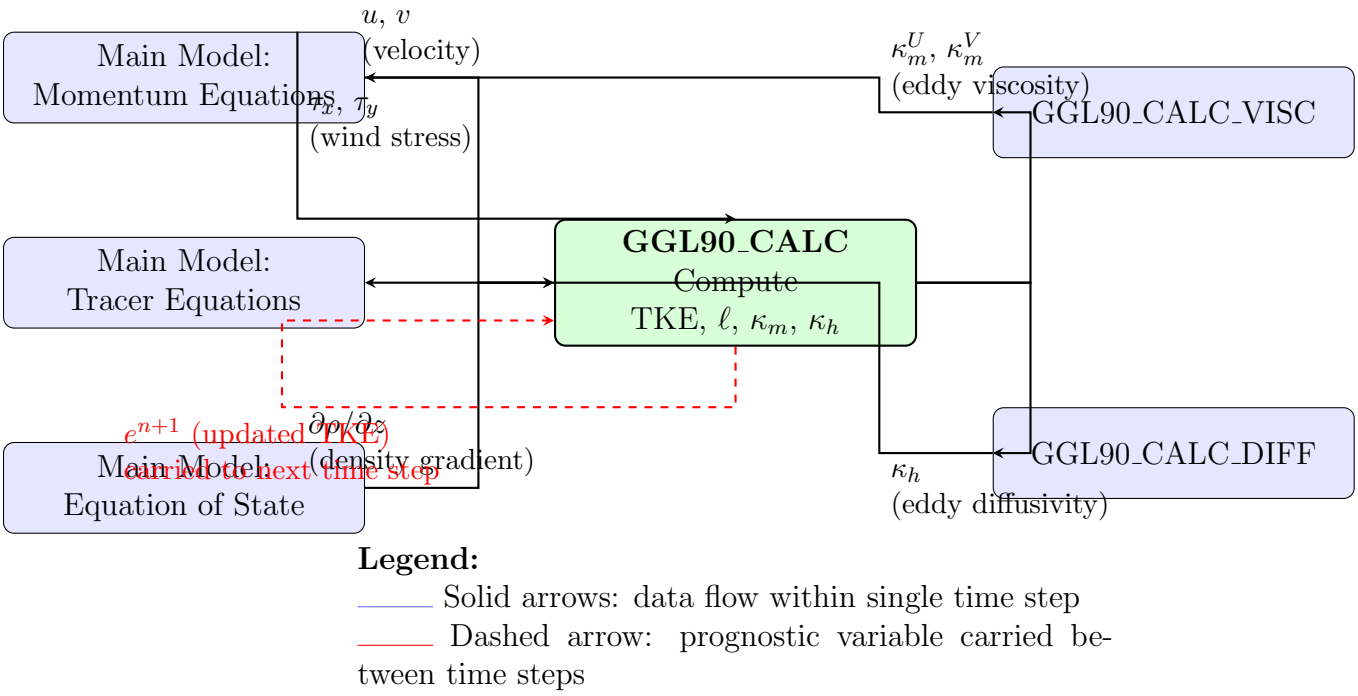

Figure 2: Data flow diagram for GGL90 package within a single MITgcm time step. The main model provides velocity, density gradient, and surface forcing to GGL90. GGL90 computes TKE, mixing length, and eddy coefficients, then returns the eddy coefficients to the momentum and tracer solvers via interface routines. TKE is the only prognostic variable, carried between time steps (dashed red arrow).

Interpretation of Figure 2:

1. **Input phase (left side):** At the beginning of the vertical mixing phase, the main model has already computed:

   - Updated velocity fields $u^{n+1/2}$, $v^{n+1/2}$ from momentum advection
   - Density gradient $\partial\rho/\partial z$ from the equation of state applied to $\theta^{n+1/2}$, $S^{n+1/2}$
   - Surface wind stress τ_x, τ_y from atmospheric forcing

   These fields are passed to `GGL90_CALC` as inputs.

2. **GGL90 computation (center):** `GGL90_CALC` performs the full TKE equation solve:

   - Compute stratification N^2 from $\partial\rho/\partial z$
   - Compute vertical shear S^2 from (u, v)
   - Compute mixing length ℓ from TKE and N^2 (Sections 9.1–9.4)
   - Update TKE: $e^{n+1} = e^n + \Delta t\,(\text{production} - \text{buoyancy} - \text{dissipation} + \text{diffusion})$ (Section 10.1)
   - Diagnose eddy coefficients: $\kappa_m = c_\mu \ell \sqrt{e^{n+1}}$, $\kappa_h = \kappa_m / Pr_t$ (Section 10.8)

3. **Output phase (right side):** The eddy coefficients are transferred to the main model:

   - `GGL90_CALC_VISC` adds κ_m^U, κ_m^V to the momentum solver's viscosity arrays
   - `GGL90_CALC_DIFF` adds κ_h to the tracer solver's diffusivity arrays
   - The main model then applies implicit vertical mixing using these coefficients

4. **Prognostic feedback (dashed red loop):** The updated TKE e^{n+1} is stored in the GGL90 common block and used as the starting point for the next time step. This is the *only* variable that couples time steps; all other fields are recomputed from scratch.

Frequency of data exchange:

- **Every time step:** Velocity, density gradient, wind stress → GGL90; eddy coefficients → main model.
- **Initialization only:** Grid parameters (`drF`, `drC`, masks), background mixing coefficients (`viscArNr`, `diffKrNrS`).
- **Restart (pickup) files:** TKE field written at checkpoint intervals, read at restart.

This clean separation of concerns—where GGL90 is a self-contained "black box" that receives state variables and returns mixing coefficients—makes it straightforward to replace GGL90 with alternative turbulence closures (e.g., KPP, Mellor-Yamada) without modifying the main model code.

Table 12: External inputs from main MITgcm model to GGL90 package

Variable Name	Source Module	Description	Units	Update Frequency
`sigmaR`	`CALC_PHI_HYD` (via `FIND_RHO`)	Vertical gradient of in-situ density or potential density reference level, used to compute buoyancy frequency N^2	kg m^{-4}	Every time step
`uVel`, `vVel`	Momentum solver	Horizontal velocity components on C-grid (U-points, V-points). Used to compute vertical shear $S^2 = (\partial u/\partial z)^2 + (\partial v/\partial z)^2$	m s^{-1}	Every time step
`surfaceForcingU`, `surfaceForcingV`	External forcing / surface boundary layer	Surface wind stress (kinematic, i.e., divided by ρ_0). Used to compute friction velocity u_* and surface TKE boundary condition	m^2 s^{-2}	Every time step
`viscArNr(k)`	`PARAMS.h`	Background vertical viscosity (depth-dependent array). GGL90 eddy viscosity is floored at this value	m^2 s^{-1}	Initialization only
`diffKrNrS(k)`	`PARAMS.h`	Background vertical diffusivity for scalars (depth-dependent array). GGL90 eddy diffusivity is floored at this value	m^2 s^{-1}	Initialization only
`drF(k)`, `drC(k)`	`GRID.h`	Vertical grid spacing arrays: `drF` = cell thickness at interfaces, `drC` = distance between cell centers	m	Initialization only
`maskC`, `maskU`, `maskV`	`GRID.h`	Land/ocean masks (0 = land, 1 = ocean) at tracer points (C), U-points, V-points	–	Initialization only
`Ro_surf`, `R_low`	`GRID.h`	Vertical position of surface and bottom (z-coords: depth in m; p-coords: pressure in Pa). Used for mixing length boundary constraints	m or Pa	Initialization only
`theta`, `salt`	Tracer solver	Potential temperature and salinity. **Not directly used by GGL90** except for optional diagnostics. Stratification is passed via `sigmaR`, not recomputed from T/S	°C, psu	Every time step

Table 13: GGL90 package outputs to main MITgcm model

Variable Name	Destination Module	Description	Units	Update Frequency
`GGL90viscArU`, `GGL90viscArV`	Momentum solver (via `GGL90_CALC_VISC`)	Eddy viscosity at U-points and V-points, including background floor. Used in vertical viscosity term $\partial/\partial z(\kappa_m \partial u/\partial z)$	$m^2\ s^{-1}$	Every time step
`GGL90diffKr`	Tracer solver (via `GGL90_CALC_DIFF`)	Eddy diffusivity for all scalars (T, S, passive tracers), including background floor. Used in vertical diffusion term $\partial/\partial z(\kappa_h \partial\theta/\partial z)$	$m^2\ s^{-1}$	Every time step

Table 14: GGL90 internal prognostic state (carried between time steps)

Variable Name	Storage Location	Description	Units
`GGL90TKE`	`GGL90.h` common block, `pickup_ggl90.*.data` file	Turbulent kinetic energy per unit mass. **Only prognostic variable** in standard GGL90 (without IDEMIX)	$m^2\ s^{-2}$
`IDEMIX_E`	`GGL90.h` common block (optional), `pickup_ggl90.*.data` file	Internal wave energy per unit mass. Prognostic variable when `ALLOW_GGL90_IDEMIX` is enabled	$m^2\ s^{-2}$